

DOSSIER DE PROJET PROFESSIONNEL

REALISATION DE L'APPLICATION WEB ET MOBILE

FOU2FOOT

PAR GAËLICK RIGOUX

SOMMAIRE

Chapitre 1 – Introduction générale.....	4
1.1. Contexte et objectifs du projet.....	4
1.2. Enjeux utilisateurs et métier.....	4
1.3. Périmètre et limites du projet.....	5
1.4. Méthode et organisation.....	5
Chapitre 2 – Vue d'ensemble technique.....	6
2.1. Stack technologique.....	6
2.2. Architecture de l'application.....	6
2.3. Qualité et sécurité.....	8
Chapitre 3 – Développement de la partie front-end sécurisée.....	9
3.1. CP1 – Installer et configurer l'environnement de travail.....	9
1) Outils et versions utilisées.....	9
2) Organisation des ressources.....	10
3) Finalités offertes par la configuration.....	10
3.2. CP2 – Maquetter une application.....	10
3.3. CP3 – Réaliser une interface utilisateur statique et adaptable.....	16
1) Pages statiques.....	16
2) Composants réutilisables.....	17
3) Présentation des pages front-end.....	18
4) Schémas d'enchaînement.....	24
5) Outils et technologies utilisés.....	25
6) Accessibilité.....	25
7) Lighthouse.....	25
3.4. CP4 – Développer une interface utilisateur dynamique.....	27
1) Navigation dynamique.....	27
2) Gestion des formulaires.....	28
3) Interaction avec le panier.....	29
4) Mises à jour côté client.....	30
5) Sécurité front-end.....	31
6) Jeu d'essai fonctionnel.....	33
3.5. Accessibilité et éco-conception.....	33
1) Principes appliqués.....	33

2) Lightscore page login.....	34
3) Compétences mobilisées.....	35
4) Éco-conception.....	36
5) Veille sécurité front-end.....	37
3.6. Bilan du front-end.....	38

ANNEXES.....39

2.2. Architecture de l'application.....	40
b) Schéma de fonctionnement.....	40
c) Schéma global d'architecture.....	41
3.1. CP1 – Étapes d'installation.....	42
3.2. CP2 – Maquettes et wireframes.....	43
3.3. CP3 – Captures pages front-end.....	49
Simulation navigation utilisateur.....	52
Scores Lighthouse détaillés.....	56
3.4. CP4 – Extraits de code.....	58

Chapitre 1 -Introduction générale

1.1. Contexte et objectifs du projet

Le projet « Fou2Foot » s’inscrit dans le cadre de ma formation de Développeur Web et Web Mobile. Il a pour objectif de mettre en pratique l’ensemble des compétences du référentiel REAC à travers la conception et le développement d’une application web moderne.

L’application a été pensée comme une boutique e-commerce spécialisée dans les maillots de football. J’ai effectué ce choix parce que je suis un client occasionnel de ce type de site. J’ai pensé m’inspirer de la logique de leur architecture pour tenter de reproduire à mon tour un site complet et fonctionnel. Par leur fréquentation j’ai également pu cerner les limites de certains. J’y voyais une promesse d’améliorations possibles.

Ce choix thématique répond aussi à un cas d’usage courant (site marchand), tout en offrant une base pédagogique suffisamment conséquente pour manipuler des concepts variés : catalogue de produits, gestion du panier, comptes utilisateurs, adresses... J’avais pour intention de créer un site permettant aux utilisateurs :

- de consulter un catalogue de clubs et de maillots.
- d’afficher les détails de chaque produit.
- d’ajouter des articles dans un panier ou dans une WishList.
- de gérer un compte utilisateur et ses adresses.
- de simuler un processus de commande + paiement.

Quant à la partie relevant de l’administrateur, celui-ci devait pouvoir gérer son profil, les comptes utilisateurs, les commandes, les stocks des maillots, créer ou modifier un club ou un maillot, consulter ses statistiques.

Ce projet nécessite la maîtrise conjointe du front-end et du back-end, ainsi que l’intégration des aspects liés à la sécurité, à l’ergonomie et à la gestion de projet.

1.2. Enjeux utilisateurs et métier

Du point de vue utilisateur, les enjeux étaient :

- Simplicité d’utilisation : navigation intuitive, interface claire, parcours d’achat fluide.
- Rapidité et réactivité : grâce à Inertia et React, qui évitent les rechargements complets de page.

- Accessibilité : compatibilité avec divers supports (desktop et mobile).
- Sécurité : gestion fiable des comptes et sessions utilisateurs, protection des données personnelles.

1.3. Périmètre et limites du projet

J'ai circonscrit dans un premier le périmètre de l'application à :

- la gestion des clubs et maillots (catalogue et fiches produits).
- l'affichage des fiches produits avec images.
- la gestion du panier (ajout, mise à jour, suppression, vidage).
- la gestion d'un compte utilisateur (authentification, adresses)

Au fil du développement, j'ai ensuite étendu le périmètre initial en implémentant les fonctionnalités suivantes :

- La simulation d'un système de paiement en ligne via Stripe.
- La mise en place du suivi des commandes.
- La possibilité pour l'utilisateur d'ajouter des maillots dans une Wishlist.
- La gestion avancée d'un espace administrateur.

1.4. Méthode et organisation

J'ai mené le projet en suivant une approche incrémentale et « agile ». J'ai procédé au découpage en fonctionnalités prioritaires (MVP¹ : catalogue, panier, compte utilisateur). J'ai dans un premier temps concentré mon effort à la mise en place des fonctionnalités de base. J'ai débuté par la création du compte utilisateur côté client (inscription , connexion, détails du compte, adresses). J'ai ensuite travaillé sur la partie catalogue pour terminer avec le panier.

Cette façon de procéder permet d'obtenir rapidement une première version utilisable. Les utilisateurs peuvent tester le site et émettre des retours. J'ai ainsi pu valider les choix technologiques et la faisabilité technique de mon projet :

- J'ai mis en place progressivement le front-end et le back-end.
- J'ai Utilisé Git pour la gestion des versions (versioning) et la collaboration (création de branches afin de procéder à des essais sans affecter le code initial).
- J'ai effectué des tests manuels réguliers pour valider chaque étape de mon projet.

¹ **MVP** (*Minimum Viable Product*, ou « produit minimum viable » en français)

La concrétisation d'objectifs simples et atteignables m'a apporté de la visibilité dans la réalisation de mon projet. Une fois le MVP achevé, je me suis consacré aux processus de paiement et de commande. J'ai conclu la partie dédiée aux produits avec la réalisation de la Wishlist. J'ai finalisé Fou2Foot avec la création de la partie dédiée à l'administrateur. Ce rôle dispose d'autorisations étendues par rapport à un utilisateur lambda, lui permettant de gérer l'ensemble du site.

Chapitre 2 -Vue d'ensemble technique

2.1. Stack technologique

Mon projet Fou2Foot repose sur une architecture monolithique **Laravel** intégrant **Inertia.js** et **React**. Cette approche me permet de bénéficier de la réactivité d'une application moderne, le front et le back cohabitent dans un même environnement applicatif.

- Laravel 11.9 (PHP 8.3.14) : framework²back-end, gestion du routage, logique métier, accès BDD via Eloquent ORM.
- Inertia.js : pont entre Laravel et React.
- React : framework front-end, création d'interfaces dynamiques et réactives.
- Tailwind CSS : framework CSS utilitaire, garantissant un design responsive et homogène.
- Vite : outil de build rapide et moderne pour le front-end.
- MySQL : système de gestion de base de données relationnelle(SGBDR).

2.2. Architecture de l'application

a) Architecture

L'architecture suit le modèle MVC (modèle, vue, contrôleur) de Laravel, enrichi par l'intégration Inertia/React :

Côté serveur (Laravel)

- Models : représentent les entités métier (User, Club, Maillot, Cart, Address).

² Framework : un framework est un ensemble de composants logiciels réutilisables qui permettent de développer de nouvelles applications plus efficacement.

- Controllors : assurent la logique métier (ex. CartController pour gérer le panier).
- Migrations : définissent la structure des tables et leurs relations.
- Routes : exposent les points d'entrée (parcours public, parcours utilisateur connecté).

Côté client (React via Inertia)

- Pages : chaque route est liée à une page React (Home, MaillotDetail, Cart, AccountDetails...).
- Composants : éléments réutilisables (Header, Footer, WelcomeMessage, PanierLink).
- Styles : Tailwind pour la mise en forme et la responsivité.

Cette organisation induit :

- une séparation claire entre logique serveur et interface,
- une expérience fluide côté utilisateur (navigation sans rechargement complet),
- une évolutivité (possibilité d'ajouter de nouvelles pages et composants facilement).

b) Schéma de fonctionnement

Le fonctionnement de l'application repose sur une chaîne de traitement en 5 étapes :

1. Requête utilisateur : lorsqu'un utilisateur accède à une page (ex. /maillots), la route Laravel correspondante est appelée via le fichier routes/web.php.
2. Traitement côté serveur : le contrôleur exécute la logique métier (récupération des données, pagination, filtrage) via les modèles Eloquent.
3. Transmission via Inertia.js : Inertia agit comme une passerelle entre Laravel et React. Les données envoyées par le contrôleur sont automatiquement transformées en props React et injectées dans le composant correspondant.
4. Rendu côté client (React) : le composant React affiche les données et gère l'interactivité (filtrage, clics, ajouts au panier). Toute action utilisateur est renvoyée vers les routes Laravel via Inertia, sans rechargement complet de page.
5. Mise à jour dynamique : Inertia ne recharge que le composant concerné, simulant un comportement SPA tout en restant dans un monolithe Laravel unique. Cela garantit une navigation fluide, la cohérence des sessions et une sécurité unifiée.

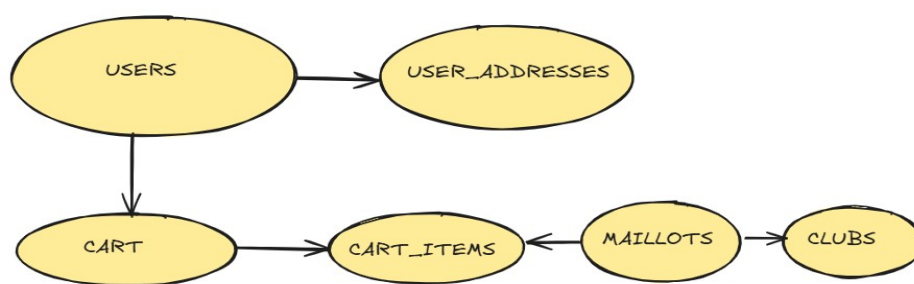
Les extraits de code correspondants sont disponibles en annexes.

c) Schéma global d'architecture

Le Front-end (React) : gère l'affichage et l'interaction utilisateur, tandis qu'Inertia.js : transmet les données entre Laravel et React sous forme de props. Le Back-end (Laravel) : applique la logique métier, valide les données et interagit avec la base de données. Quant à la Base de données (MySQL), elle stocke les informations persistantes (utilisateurs, clubs, maillots, adresses, paniers, commandes, WishList).

Voir schéma en annexes

d) Schéma de base de données (simplifié)



Explication des relations :

Un User peut avoir plusieurs Adresses (relation N-N via user_addresses).

Un User possède un Cart (panier).

Un Cart contient plusieurs CartItems (articles panier), chacun lié à un Maillot.

Un Maillot appartient à un Club.

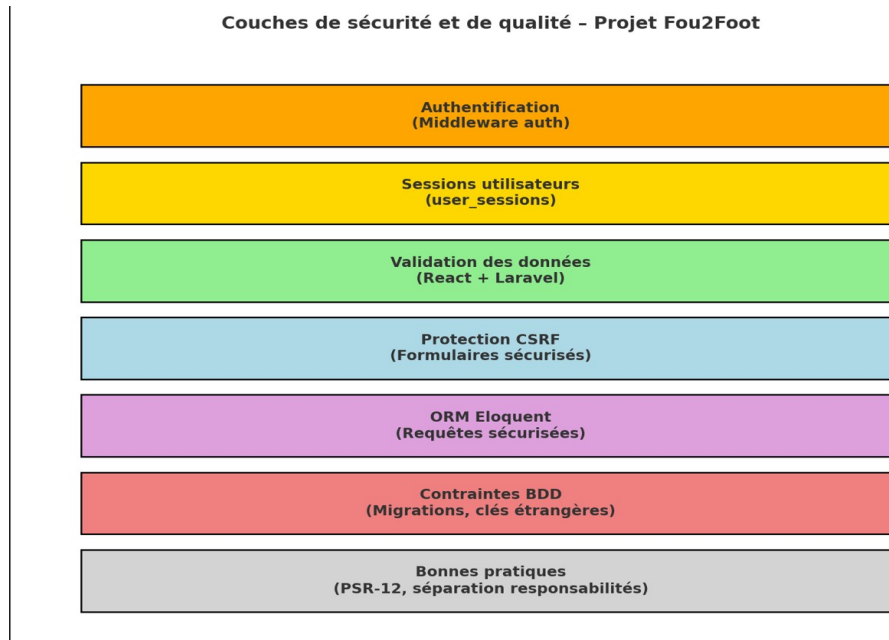
2.3. Qualité et sécurité

La sécurité fait partie des enjeux prioritaires de la conception d'une application. Plusieurs mécanismes intégrés à Laravel et aux outils front-end m'ont aidé à assurer la fiabilité du système

- **Validation des données** : les formulaires sont systématiquement validés côté serveur (Laravel) et côté client (React). Cela réduit les risques d'erreurs et empêche l'injection de données malveillantes.

- **Protection CSRF** : elle est activée par défaut dans Laravel ; elle sécurise tous les formulaires en générant un jeton unique associé à chaque session.

- **Gestion des relations en base** : les contraintes d'intégrité référentielle (clés primaires et étrangères) sont mises en place via les migrations, garantissant la cohérence des données.



Chapitre 3 -Développement de la partie front-end sécurisée

3.1. CP1 -Installer et configurer l'environnement de travail

1) Outils et versions utilisées

Outil / Technologie	Version
PHP	8.3.14
Laravel	11.9
Node.js	20.17.0
npm	10.8.2
React	18.3.1
React-DOM	18.3.1
TailwindCSS	3.4.10
Vite	5.0
MySQL	8.0
Inertia.js	1.3.0

(étapes d'installations voir annexes)

2) Organisation des ressources

- ressources/js/Pages : pages React (ex. MaillotsList.jsx, MaillotDetail.jsx).
- ressources/js/Components : composants réutilisables (boutons, formulaires, liens).
- ressources/css : styles globaux centralisés.

3) Finalités offertes par la configuration

Grâce à cette configuration, j'ai pu :

- développer efficacement avec **React + Tailwind** sans surcharge CSS.
- profiter de la navigation fluide via **Inertia.js**.
- compiler et tester rapidement grâce à **Vite** (HMR :Hot Module Replacement) : avec **Vite**, dès la modification d'un fichier (JS/TS/React/Tailwind), celui-ci **réinjecte** uniquement le module modifié dans le navigateur **sans recharger toute la page**. Cela me permet d'obtenir un cycle de développement ultra-rapide avec un état des composants conservé et un retour immédiat.

3.2. CP2 -Maquetter une application

La phase de conception visuelle a été réalisée afin de structurer l'interface et de préparer le travail front-end. Cette étape a suivi un processus progressif allant du **zoning à la maquette**.

- **Le zoning** est la représentation schématique de la page en zones fonctionnelles (header, menu, contenu, footer). L'objectif est d'organiser les blocs principaux sans se soucier du design graphique.
- **le wireframe** constitue une version plus détaillée du zoning. On affine et on place les éléments sans couleur ni style définitif. Le wireframe précise la disposition et les interactions prévues.
- La **maquette** est la représentation graphique finalisée, incluant couleurs, polices, images et styles. C'est une projection fidèle du rendu attendu dans l'application.

J'ai suivi les étapes de conception préconisées ci-dessus. J'ai débuté la réalisation de mon maquettage par un zoning en utilisant Figma. Avec le Zoning, j'ai identifié les zones principales de l'application à savoir une zone header (logo + navigation), une zone dédiée au contenu (liste des maillots, fiches produits, panier), une zone footer (informations légales, contacts).

Avec le Wireframe (voir annexes), j'ai réalisé des croquis fonctionnels (toujours sur Figma) pour représenter la disposition des éléments. Je me suis inspiré à ce stade de plusieurs sites de vente en ligne de maillots de football notamment le site footbebe. J'ai repris l'enchaînement des pages, les éléments à insérer dans le tableau de bord, la page d'accueil, les pages du catalogue, la page détail d'un maillot.

Avec les maquettes Figma j'ai enfin créé des interfaces graphiques intégrant la palette de couleurs choisie (liée à l'univers du football), la typographie adaptée à une lecture claire et dynamique, les icônes et composants réutilisables (boutons, formulaires).

J'ai réfléchi à la maquette en amont afin de me guider dans le développement. Elle couvre deux périmètres distincts : l'interface client (parcours d'achat) et l'interface administrateur (gestion du back-office).

Parcours client :

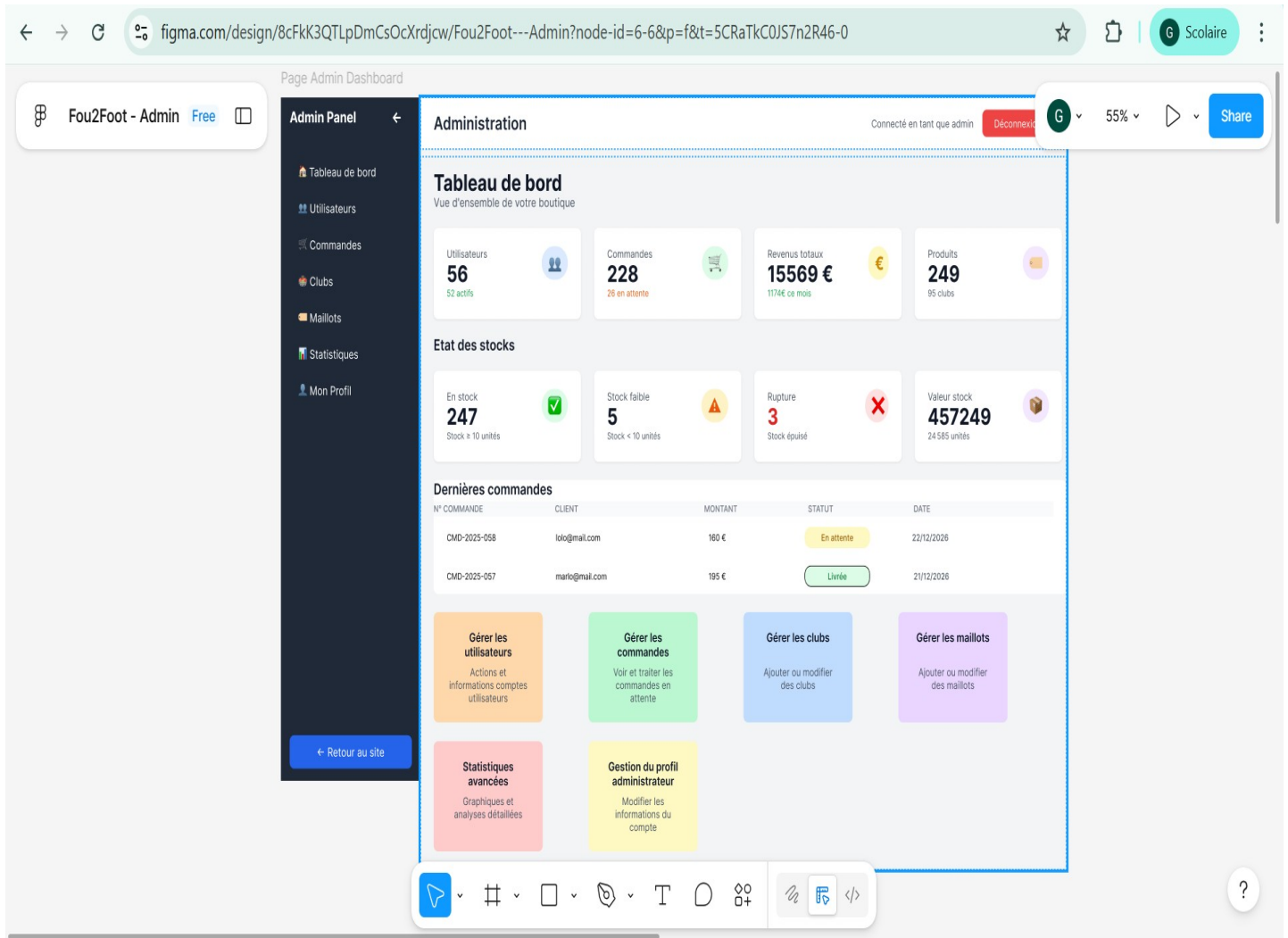
- **Accueil** : vitrine du site, accès rapide au catalogue.
- **Catalogue des maillots** : affichage en grille, filtrage par clubs.
- **Fiche produit** : visuels du maillot, description, prix, disponibilité, personnalisation du produit, bouton "ajouter au panier".
- **Panier** : liste des articles ajoutés, quantités modifiables, total calculé automatiquement.
- **Authentification** : page de connexion et d'inscription.
- **Espace utilisateur/ tableau de bord** : gestion des adresses, suivi des commandes.
- **WishList**: l'utilisateur connecté ou non peut ajouter un maillot à une WishList.
- **Paiement** : simulation de paiement.
- **Commandes** : affichage des commandes.

Interface administrateur (exemple principal de maquettage) :

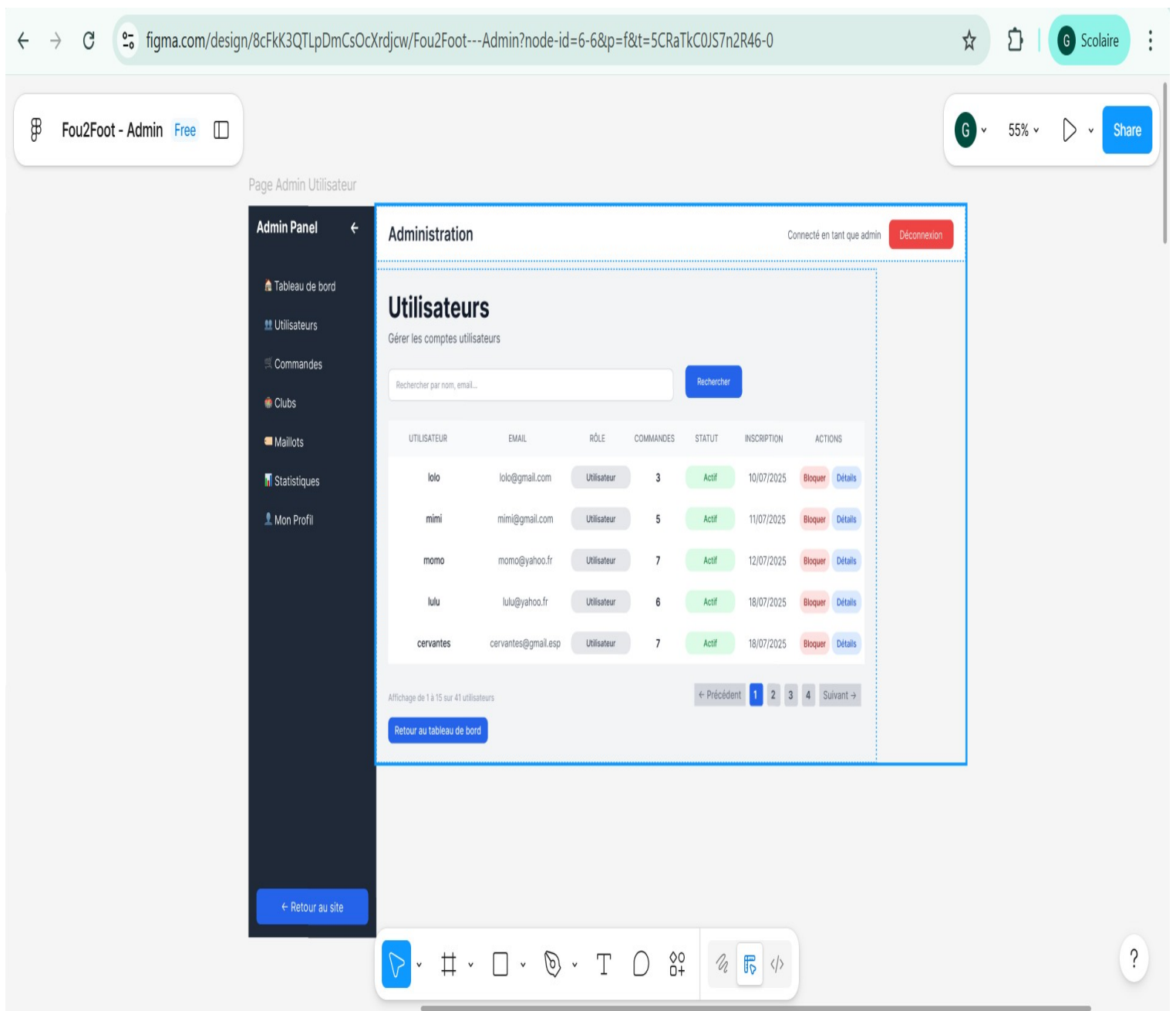
L'espace d'administration a fait l'objet d'un maquettage complet et structuré sous Figma. Il constitue l'exemple le plus représentatif de la démarche de conception, car il intègre simultanément un design system cohérent (composants réutilisables, couleurs, typographie), une navigation complexe (sidebar fixe, header) et plusieurs types d'interfaces différentes.

Les pages suivantes ont été maquettées :

AdminDashboard : vue synthétique avec indicateurs clés (chiffre d'affaires, commandes, utilisateurs actifs, stock faible), graphiques de tendance.

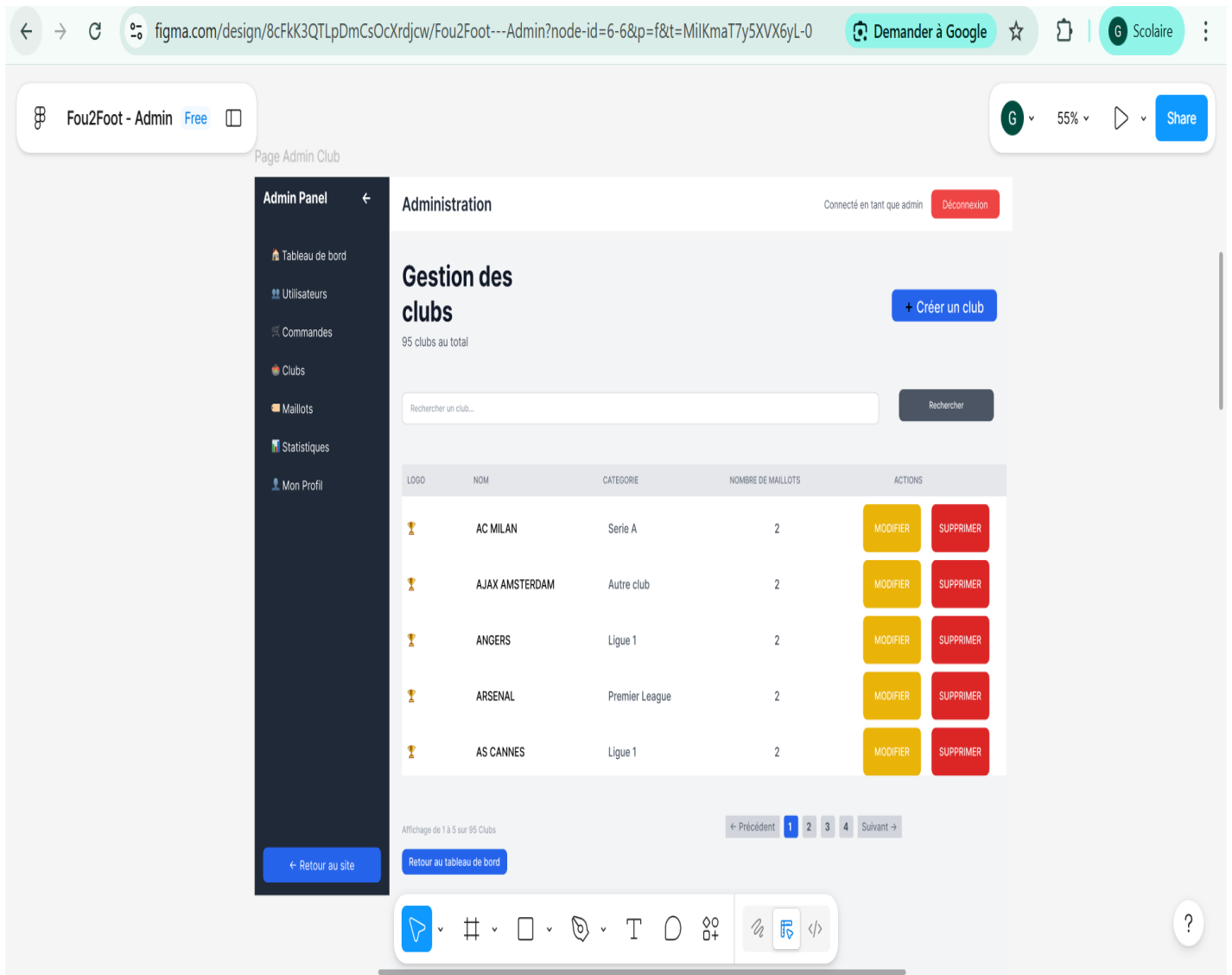


Gestion des utilisateurs : tableau filtrable avec actions (Bloquer, détails), badges de rôle et de statut.



Gestion des commandes : tableau avec statuts colorés (en attente, expédiée, livrée, annulée), filtres par date et recherche. (voir maquette en annexes)

Gestion des clubs : liste des clubs avec logo, ligue, nombre de maillots, actions CRUD.



Modale Créer un club

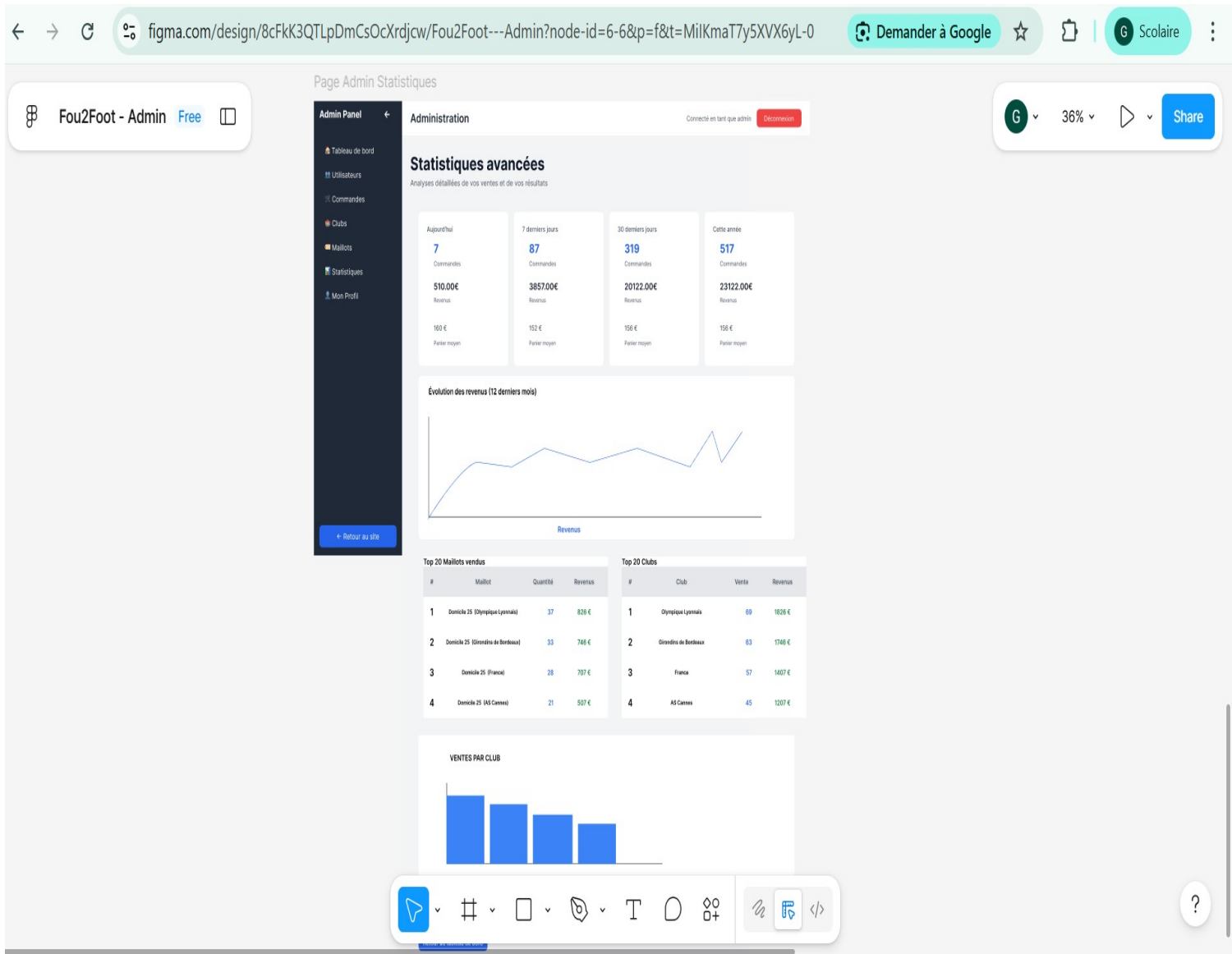
The modal form 'Créer un club' contains the following fields and controls:

- Nom du club**: A text input field with the placeholder 'Placeholder'.
- Catégorie**: A dropdown menu currently showing 'Tous les clubs'.
- Logo**: A file selection input with the placeholder 'Choisir un fichier'.
- Buttons**: 'Annuler' (grey) and 'Créer' (blue).

Dimensions: 448 x 377 Hug

Gestion des maillots : catalogue admin avec prix, stock, actions de modification. (voir maquette en annexes)

Statistiques : graphiques de revenus, top 20 maillots et clubs, indicateurs par période.



Profil administrateur : informations personnelles et modification du mot de passe. (voir maquette en annexes)

L'outil de prototypage Figma m'a permis de concevoir un design system complet en amont du développement : palette de couleurs (#2563EB pour les actions, #1E293B pour la sidebar), typographie (Inter), composants réutilisables (badges, boutons, inputs, tableaux, modales), et schéma d'enchaînement des pages via un système de navigation latérale persistante. Les pages de détail (AdminUsersShow.jsx et AdminOrdersShow.jsx) ont été développées dans la continuité de leurs pages Index respectives, en suivant le même design system.

3.3. CP3 -Réaliser une interface utilisateur statique et adaptable

Après la phase de maquettage, l'étape suivante a consisté à transformer ces maquettes en interfaces statiques fonctionnelles, en respectant les principes d'accessibilité, de responsive design et de cohérence graphique définis lors de la conception. Cette étape correspond à la mise en œuvre concrète des maquettes, sans intégrer encore la logique dynamique ou les traitements métier.

1) Pages statiques :

L'application se compose de deux ensembles de pages statiques : les pages destinées aux utilisateurs et les pages dédiées à l'administration.

Pages client :

- Home.jsx (page d'accueil qui correspond à la vitrine du site, accessible hors connexion.
- LoginRegister.jsx et Register.jsx : connexion et inscription d'un nouvel utilisateur
- Dashboard.jsx : Tableau de bord de l'utilisateur connecté.
- Addresses.jsx : gestion des adresses de facturation et livraison.
- AccountDetails.jsx : détails et modification du compte utilisateur.
- MaillotsList.jsx : catalogue filtré par club.
- MaillotDetail.jsx : fiche détaillée d'un maillot avec personnalisation.
- Panier.jsx : produit sélectionné par l'utilisateur.
- Checkout.jsx : processus de commande en plusieurs étapes.
- Payment.jsx : Page de paiement
- OrderConfirmation.jsx : confirmation de commande après paiement, email de confirmation de commande envoyé.

- Contact.jsx : page de contact.
- MyWishList.jsx: maillots ajoutés par l'utilisateur dans une liste mémorisée.
- Legal.jsx, Privacy.jsx, Terms.jsx, Returns.jsx, Delivery.jsx, CategoryPage.jsx , Order.jsx

Pages administrateur :

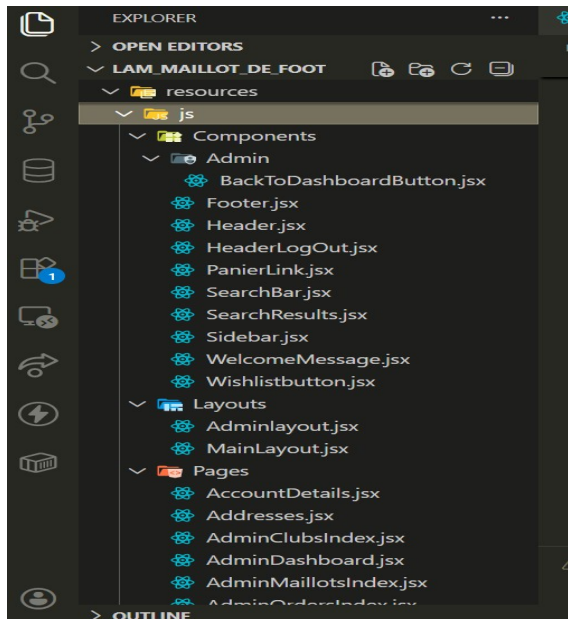
- AdminDashboard.jsx : tableau de bord avec Indicateurs clés de performance (chiffre d'affaires, commandes du jour, utilisateurs actifs, alertes de stock faible).
- AdminUsersIndex.jsx : liste paginée des utilisateurs avec filtres, badges de rôle (Administrateur / Client) et de statut (Actif / Inactif).
- AdminUsersShow.jsx : détail d'un utilisateur.
- AdminOrdersIndex.jsx : gestion des commandes avec badges de statut colorés et modale de détail.
- AdminOrdersShow.jsx : détail d'une commande.
- AdminClubsIndex.jsx : gestion des clubs (création, modification, suppression).
- AdminMaillotsIndex.jsx : gestion du catalogue de maillots avec gestion du stock.
- AdminStatistics.jsx : statistiques de vente, top 20 maillots et clubs, graphiques.
- AdminProfile.jsx : profil et sécurité du compte administrateur.

2) Composants réutilisables :

- Header.jsx et Footer.jsx pour la navigation,
- WelcomeMessage.jsx : affiche le nom et l'email de l'utilisateur dans le tableau de bord.
- Sidebar.jsx pour avoir accès au dashboard sur les pages de l'utilisateur.
- PanierLink.jsx pour le résumé du panier accessible depuis toutes les pages.
- SearchResults.jsx (pour la barre de recherche).
- WishListButton.jsx : pour ajouter un maillot dans la WishList.
- BackToDashboard.jsx : lien de retour vers le tableau de bord, réutilisé sur les pages secondaires.

Layouts (resources/js/Layouts/) :

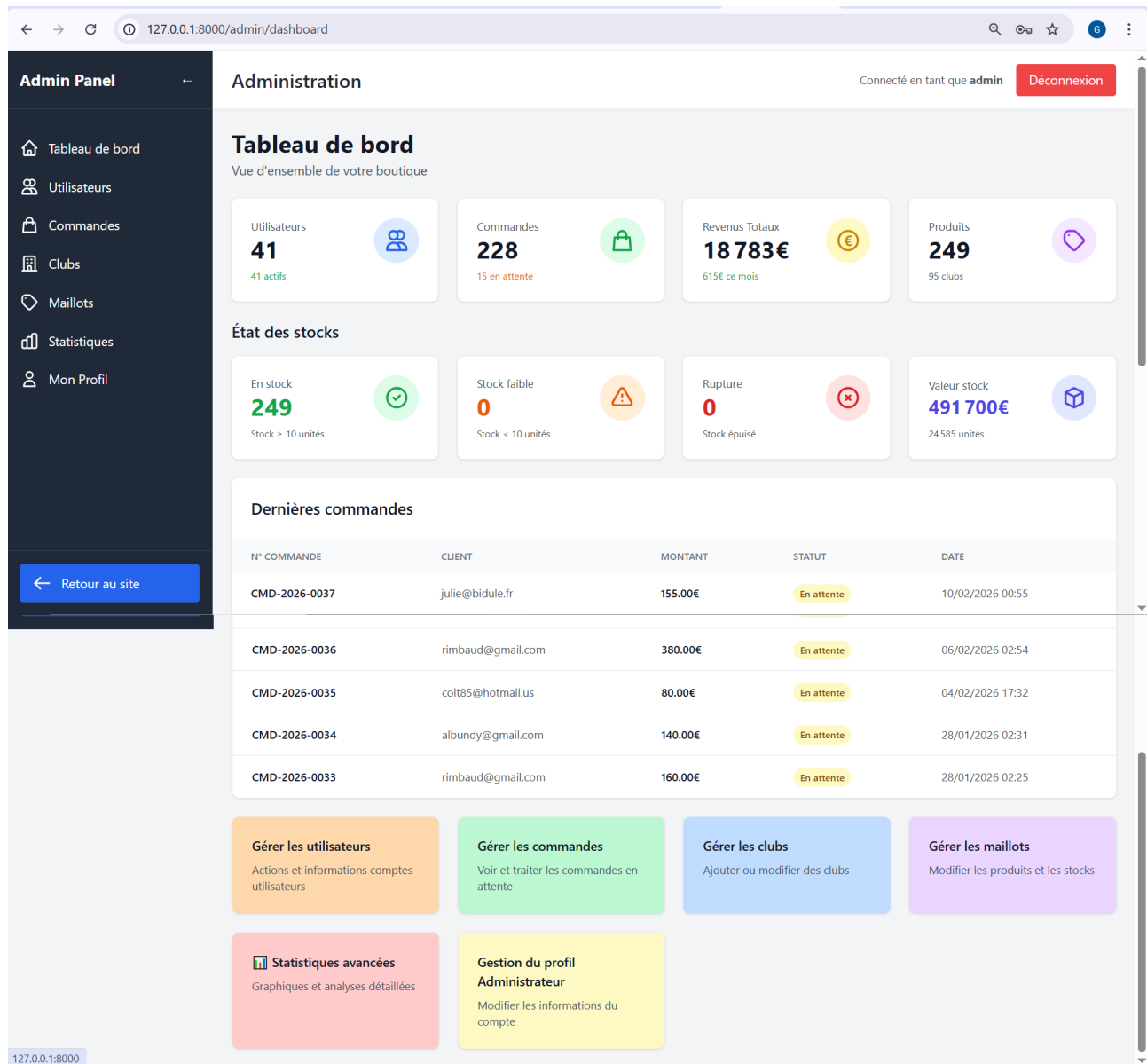
- MainLayout.jsx : gabarit principal des pages client (header, footer, contenu).
 - Adminlayout.jsx : gabarit dédié à l'administration, intégrant la sidebar admin et le header admin.
- Toutes les pages d'administration héritent de ce layout, ce qui garantit une cohérence visuelle et structurelle sur l'ensemble du back-office.



3) Présentation des pages front-end

Toutes les pages sont responsive grâce à Tailwind CSS.

Tableau de bord Administration



AdminUsersIndex.jsx

The screenshot shows a web application interface for managing users. The browser address bar displays "127.0.0.1:8000/admin/users". The page has a dark sidebar on the left with the "Admin Panel" header and a list of navigation items: "Tableau de bord", "Utilisateurs", "Commandes", "Clubs", "Maillots", "Statistiques", and "Mon Profil". The main content area is titled "Administration" and "Utilisateurs", with a subtitle "Gérer les comptes utilisateurs". It indicates that the user is logged in as "admin" and provides a "Déconnexion" button. A search bar labeled "Rechercher par nom, email..." is present, followed by a "Rechercher" button. Below the search bar is a table listing users with columns: ID, UTILISATEUR, EMAIL, RÔLE, COMMANDES, STATUT, INSCRIPTION, and ACTIONS. The table contains six rows of user data. At the bottom of the sidebar is a "Retour au site" button.

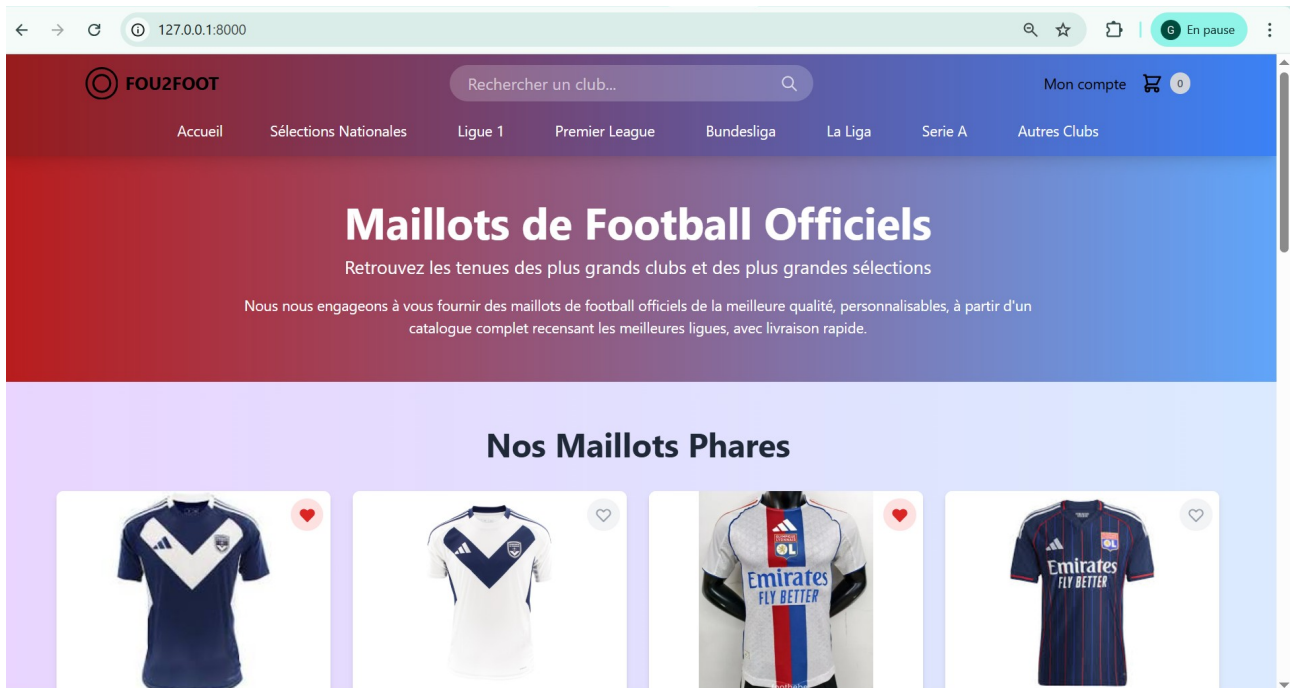
ID	UTILISATEUR	EMAIL	RÔLE	COMMANDES	STATUT	INSCRIPTION	ACTIONS
#43	duncan	duncan@free.fr	Utilisateur	0	Actif	11/02/2026	Bloquer Détails
#42	julie julie tartempion	julie@bidule.fr	Utilisateur	1	Actif	10/02/2026	Bloquer Détails
#41	marlo Marlo Stanfield	marlo@hotmail.us	Utilisateur	0	Actif	04/02/2026	Bloquer Détails
#40	fuentes Carlos Fuentes	carlos.fuentes@ph.mex	Utilisateur	0	Actif	04/02/2026	Bloquer Détails
#39	philou	philou@gmail.com	Utilisateur	0	Actif	04/02/2026	Bloquer Détails
#38	elise	elise@hotmail.fr	Utilisateur	0	Actif	04/02/2026	Bloquer Détails

Page AdminClub

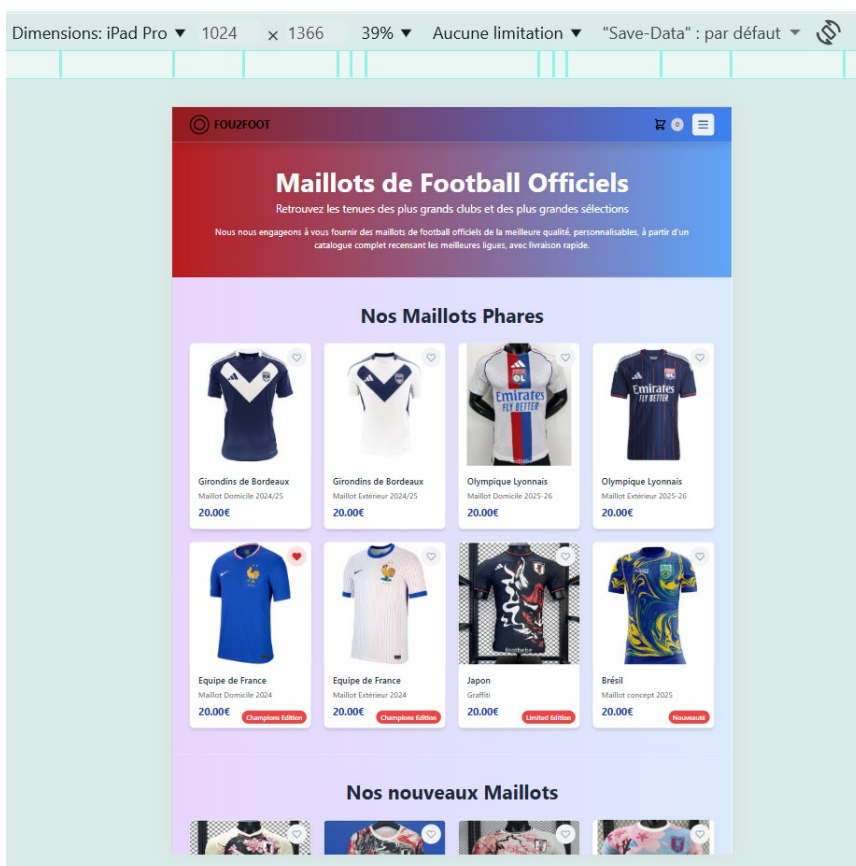
The screenshot shows a web application interface for managing clubs. The browser address bar displays "127.0.0.1:8000/admin/clubs". The page has a dark sidebar on the left with the "Admin Panel" header and a list of navigation items: "Tableau de bord", "Utilisateurs", "Commandes", "Clubs", "Maillots", "Statistiques", and "Mon Profil". The main content area is titled "Administration" and "Gestion des clubs", with a subtitle "95 club(s) au total". It indicates that the user is logged in as "admin" and provides a "Déconnexion" button. A search bar labeled "Rechercher un club..." is present, followed by a "Rechercher" button. Below the search bar is a table listing clubs with columns: LOGO, NOM, CATÉGORIE, NOMBRE DE MAILLOTS, and ACTIONS. The table contains five rows of club data. At the bottom of the sidebar is a "Retour au site" button.

LOGO	NOM	CATÉGORIE	NOMBRE DE MAILLOTS	ACTIONS
	AC Milan	Serie A	2	Modifier Supprimer
	Ajax Amsterdam	Autres clubs	2	Modifier Supprimer
	Angers SCO	Ligue 1	2	Modifier Supprimer
	Annecy FC	Ligue 1	2	Modifier Supprimer
	Arsenal	Premier League	2	Modifier Supprimer

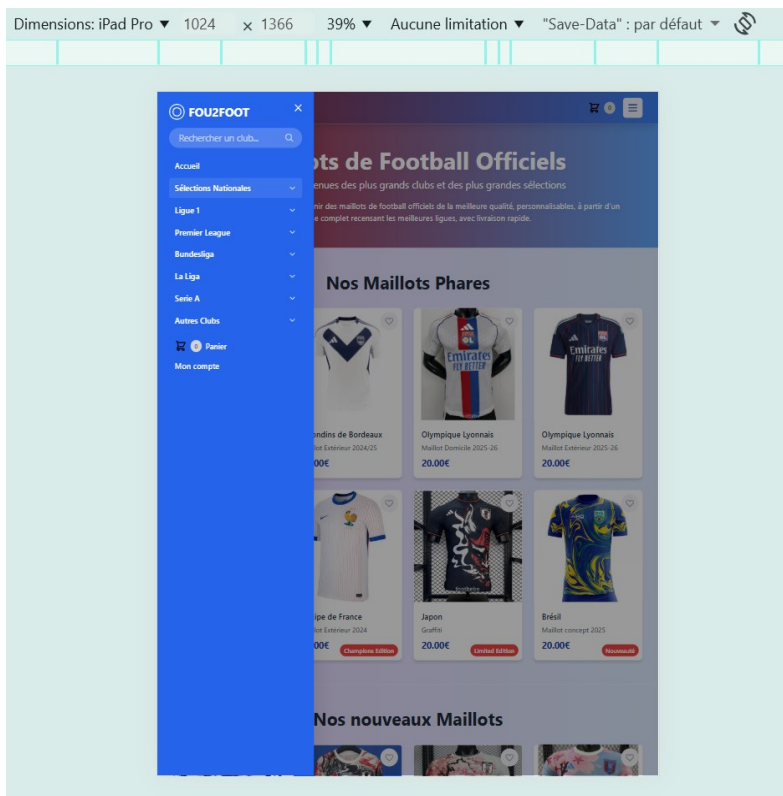
Page Utilisateur Accueil vue Desktop



Design responsive



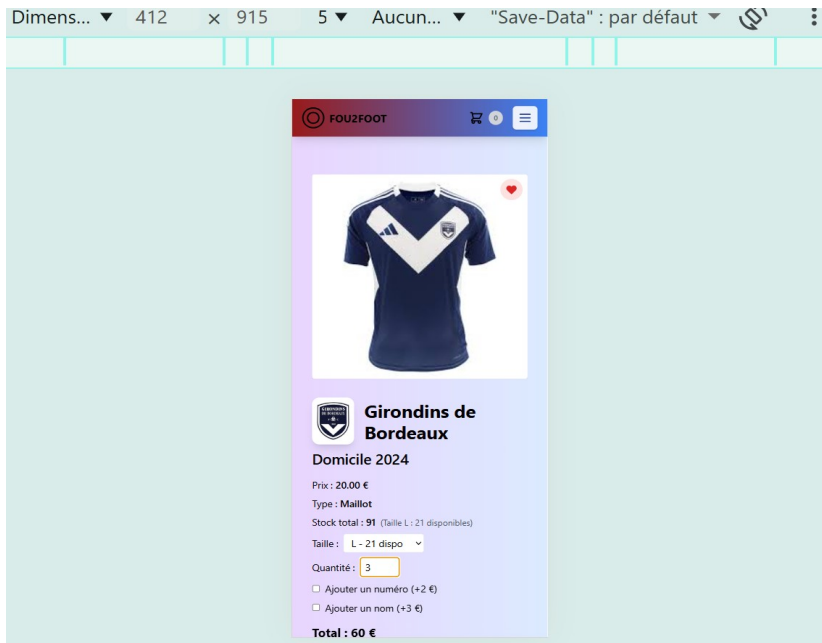
Design responsive avec menu burger



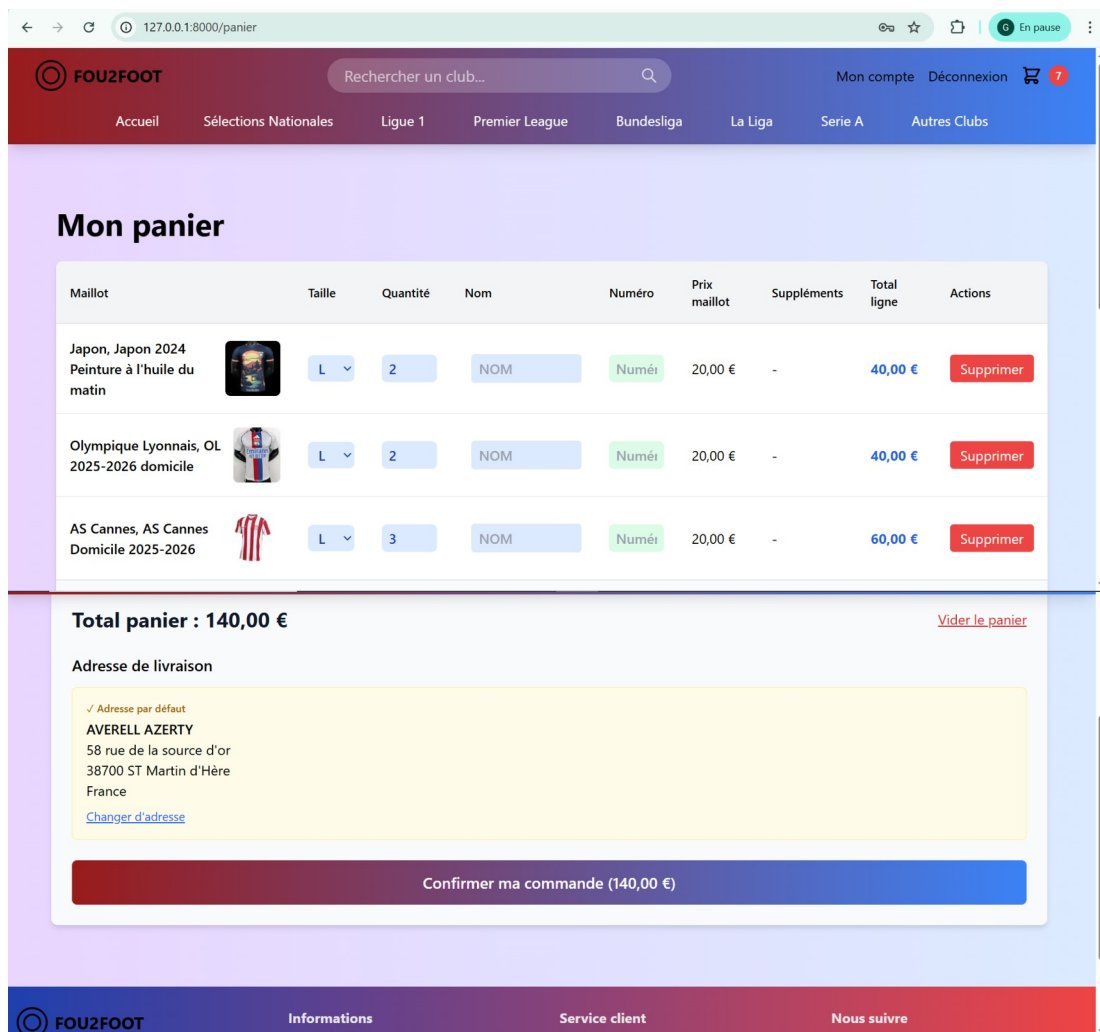
Page Utilisateur Desktop MaillotDetail.jsx



Vue Responsive avec modèle Pixel 7



Page Utilisateur Desktop Mon Panier



* Voir annexes pour le reste des pages front-end et la simulation de navigation par un utilisateur à partir de ces pages

4) Schémas d'enchaînement

Schéma d'enchaînement Admin

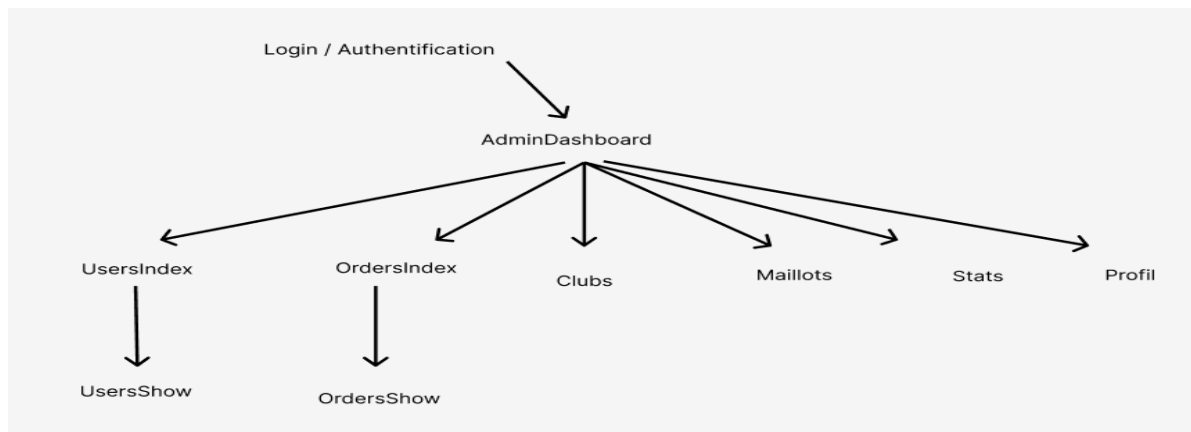
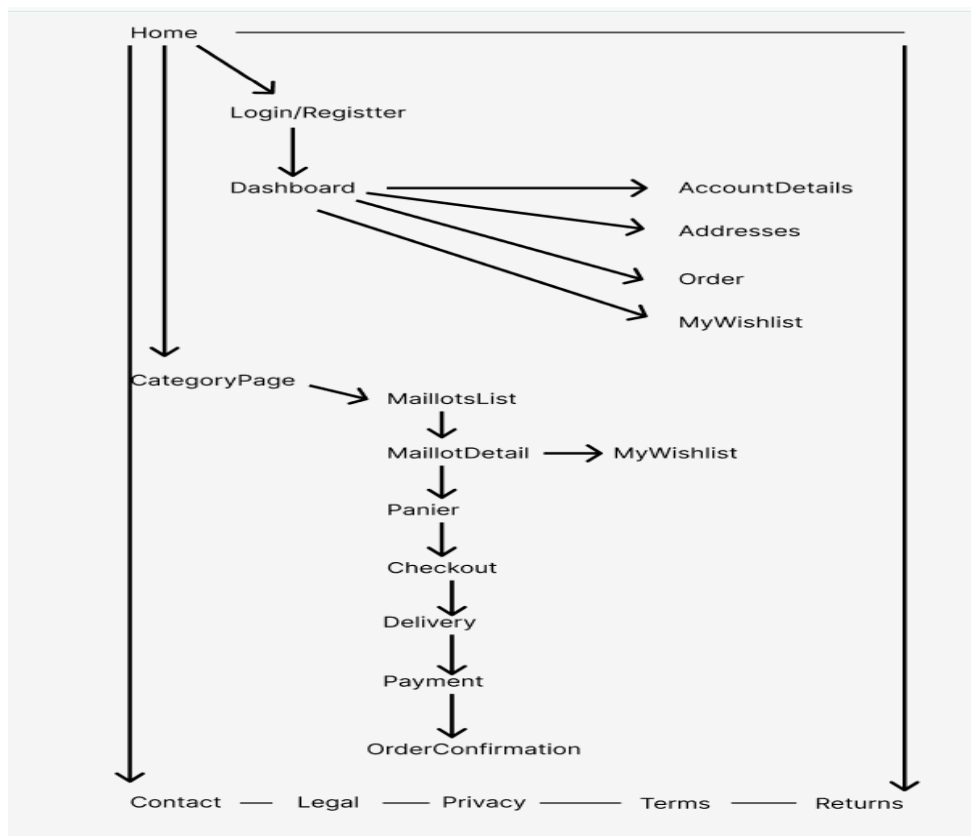


Schéma d'enchaînement Utilisateur



5) Outils et technologies utilisés

- **React 18.3.1** pour la création de composants modulaires et réutilisables.
- **TailwindCSS 3.4.10** pour appliquer rapidement une mise en forme responsive et homogène. TailwindCSS permet aux classes utilitaires (flex, grid, p-4, md:w-1/2, etc.) de garantir un affichage adapté sur mobiles, tablettes et ordinateurs.
- **Inertia.js 1.3.0** pour intégrer les vues React au sein du projet Laravel sans rechargement complet de la page.

6) Accessibilité :

Utilisation de balises sémantiques (<nav>, <main>, <footer>), respect du contraste entre texte et fond, champs de formulaires correctement labellisés avec **aria label**.

Exemple dans C:\wamp64\www\Lam_Maillot_de_foot\resources\js\Pages\LoginRegister.jsx

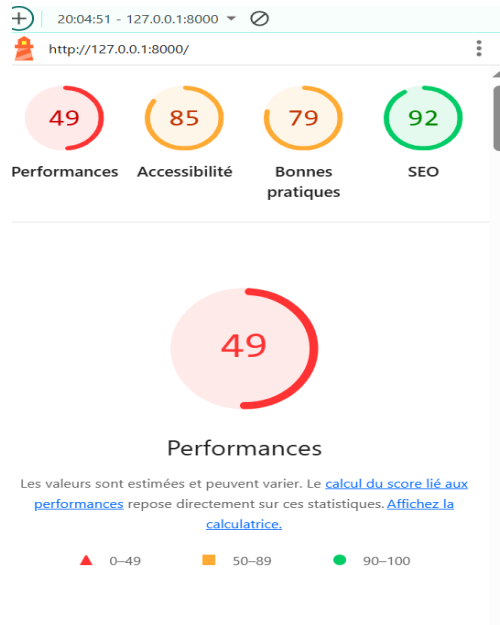
```
152 <button
153   type="button"
154   onClick={() => setShowConfirm(!showConfirm)}
155   className="absolute inset-y-0 right-0 px-3 flex items-center"
156   aria-label={showConfirm ? "Masquer la confirmation du mot de passe" : "Afficher la confirmation"}
157 >
158   <EyeIcon show={showConfirm} />
159 </button>
```

7) Lighthouse

Un audit **Lighthouse** a été réalisé via Google Chrome sur la page d'accueil de l'application (<http://127.0.0.1:8000/>), ainsi que sur la page connexion/ inscription (<http://127.0.0.1:8000/login>). L'audit est destiné à évaluer la qualité technique et l'accessibilité de l'interface. L'audit a été exécuté en environnement local (mode développement, sans optimisation de build ni compression serveur).

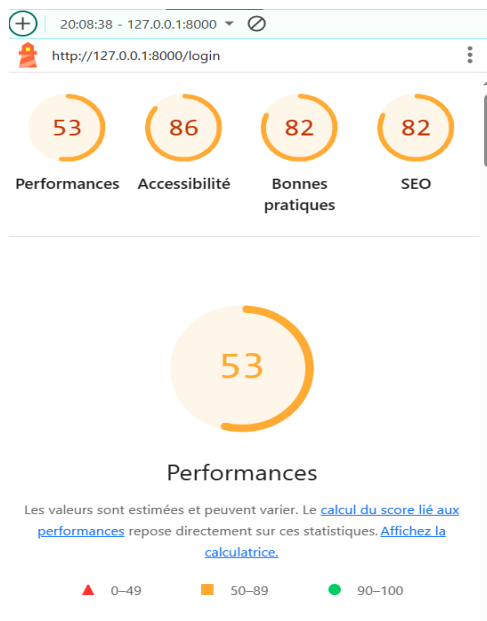
Les scores obtenus sont les suivants :

Score lighthouse accueil



Interprétation et axes d'amélioration page accueil (voir annexes)

Score lighthouse connexion/ inscription



Interprétation et axes d'amélioration page connexion /inscription (voir annexes)

3.4. CP4 -Développer une interface utilisateur dynamique

Après la mise en place des interfaces statiques, cette étape a consisté à rendre les pages interactives et connectées aux données réelles de l'application. L'objectif était d'offrir une expérience fluide, intuitive et réactive, en exploitant les capacités combinées de React et Inertia.js. Les composants gèrent leur état local, orchestrent les requêtes asynchrones vers Laravel, affichent des messages d'erreur cohérents entre le front et le back, et respectent les principes de sécurité front-end (validation, CSRF, XSS).

1) Navigation dynamique

Grâce à **Inertia.js**, la navigation entre les pages s'effectue **sans rechargement complet du navigateur**, tout en conservant la logique et la sécurité Laravel côté serveur.

Exemple : un clic sur un maillot dans MaillotsList.jsx déclenche l'appel suivant :

```
42 <Link
43   href={` /maillots/${maillot.id}`}
44   className="block group"
45   aria-label={` Voir les détails du maillot ${maillot.nom}`}
46 >
47   <div className="aspect-square overflow-hidden">
48     <img
49       src={` /${maillot.image}`}
50       alt={` Maillot ${maillot.nom} - ${club.name}`}
51       className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-300"
52       loading="lazy"
53     />
54   </div>
55
56   <div className="p-4">
57     <h2 className="font-semibold text-lg text-gray-900 mb-2 group-hover:text-blue-600 transition-colors">
58       {maillot.nom}
59     </h2>
60
61     {maillot.type && (
62       <p className="text-sm text-gray-600 mb-2">
63         {maillot.type}
64       </p>
65     )}
66
67     {maillot.prix && (
68       <p className="text-lg font-bold text-green-600">
69         {maillot.prix} €
70       </p>
71     )}
72
73     <div className="mt-3 flex items-center text-sm text-blue-600 group-hover:text-blue-800">
74       <span>Voir les détails</span>
75       <svg
76         className="ml-2 w-4 h-4 group-hover:translate-x-1 transition-transform"
77         fill="none"
78         stroke="currentColor"
79         viewBox="0 0 24 24"
80         aria-hidden="true"
81       >
82         <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9 517 7-7 7" />
83       </svg>
84     </div>
85   </div>
86 </Link>
```

La route correspondante est définie dans routes/web.php (voir annexes).

Et la méthode show() du MaillotController prépare les données avant l'envoi à React (voir annexes).

Ainsi, le passage entre la liste des produits et la fiche détail s'effectue instantanément, sans rupture de navigation.

2) Gestion des formulaires

Les formulaires d'authentification utilisent Inertia useForm : l'état du formulaire, l'envoi et les erreurs sont gérés de manière idiomatique, sans validation manuelle ad-hoc côté front. Côté serveur, Laravel valide les champs ; Inertia remonte ensuite les erreurs automatiquement dans errors, affichées au plus près des inputs.

État et soumission avec useForm

```
12  ✓  const { data, setData, post, processing, errors } = useForm({
13      login: "",
14      username: "",
15      email: "",
16      password: "",
17      password_confirmation: "",
18  })
19
20  ✓  const handleChange = (e) => {
21      setData(e.target.name, e.target.value)
22  }
23
24  ✓  const handleSubmit = (e) => {
25      e.preventDefault()
26      post(isLogin ? "/login" : "/register")
27  }
28
```

Affichage des erreurs et désactivation du bouton

```
100  ✓  {isLogin && (
101  ✓      <div>
102      <label htmlFor="login" className="block text-sm font-medium text-gray-700">Nom d'utilisateur ou Email</label>
103      <input
104  ✓          id="login"
105          name="login"
106          value={data.login}
107          onChange={handleChange}
108          required
109          className="w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm mt-1"
110      />
111      {errors.login && <p className="text-sm text-red-600">{errors.login}</p>}
112      </div>
113  ✓  )}
```

Accessibilité et UX intégrées : contrôle de la visibilité du mot de passe, focus ring, et labels explicites dans le formulaire (voir ci-dessus)

Bilan

La validation front repose sur useForm (gestion d'état, envoi, processing) et l'affichage automatique des erreurs renvoyées par Laravel (errors). La validation back reste la source de vérité (règles côté contrôleurs/FormRequest). Ce couplage garantit une UX fluide (sans rechargement) et une cohérence stricte des règles métier et sécurité.

3) Interaction avec le panier

L'ajout au panier se fait depuis la page MaillotDetail.jsx. Lorsque l'utilisateur choisit la taille et la quantité, le composant envoie une requête Inertia vers Laravel :

```
21 function handleAddToCart() {
22   router.post('/cart/add', {
23     maillot_id: maillot.id,
24     size: taille,
25     quantity: qte,
26     numero: personnalisation.numero ? numero : null,
27     nom: personnalisation.nom ? nom : null,
28   }, {
29     onSuccess: () => alert("Maillot ajouté au panier !"),
30   });
31 }
32
```

Cette requête appelle la route définie dans routes/web.php (voir annexes)

Et le contrôleur CartController.php crée ou met à jour l'article dans la base de données.

L'utilisateur voit immédiatement le panier mis à jour, sans rechargement complet, grâce à **Inertia.js**.

Le composant Panier.jsx s'appuie ensuite sur ces données pour calculer et afficher le total en temps réel :

```
// Total global du panier (somme des totaux lignes)
const prixTotal = cartItems.reduce((sum, item) => sum + (item.total || 0), 0)
```

Et un composant global `PanierLink.jsx` maintient le compteur dynamique :

```
6 export default function PanierLink() {
7   const [cartCount, setCartCount] = useState(0)
8   const [isLoading, setIsLoading] = useState(false)
9   const { auth } = usePage().props
```

4) Mises à jour côté client

La page `MaillotDetail.jsx` gère dynamiquement l'état des champs (taille, quantité, nom, numéro) et le calcul du total en temps réel. L'objectif que je m'étais fixé est de permettre à l'utilisateur de personnaliser son maillot et d'obtenir instantanément le prix mis à jour, sans rechargement de la page.

Le composant exploite les hooks React `useState` pour suivre l'état local de chaque champ et recalculer le total en fonction des choix effectués.

Exemple (`MaillotDetail.jsx`)

```
6 export default function MaillotDetail({ maillot, tailles, quantite, prix, prix_numero, prix_nom }) {
7   const [taille, setTaille] = useState(tailles[0]);
8   const [qte, setQte] = useState(1);
9   const [numero, setNumero] = useState("");
10  const [nom, setNom] = useState("");
11  const [personnalisation, setPersonnalisation] = useState({ numero: false, nom: false });
12
13  // Calcul du supplément pour personnalisation
14  const supplement =
15    (personnalisation.numero && numero ? prix_numero : 0) +
16    (personnalisation.nom && nom ? prix_nom : 0);
17
18  // Multiplie bien par la quantité choisie
19  const total = (prix + supplement) * parseInt(qte || 1, 10);
20
```

Chaque interaction (ex. changement de taille, de quantité ou activation d'une personnalisation) provoque une **mise à jour immédiate du rendu**. Le composant React réévalue `total` à chaque modification de l'état, garantissant une expérience fluide et cohérente.

Interaction utilisateur

Le champ de quantité ou les cases à cocher déclenchent une modification directe de l'état :

```
53 <div className="mb-2">
54   Quantité :
55   <input
56     type="number"
57     min="1"
58     max={quantite}
59     value={qte}
60     onChange={e => setQte(e.target.value)}
61     className="ml-2 border rounded px-2 py-1 w-16"
62   />
63 </div>
64 <div className="mb-2">
65   <label>
66     <input
67       type="checkbox"
68       checked={personnalisation.numero}
69       onChange={e => setPersonnalisation(p => ({ ...p, numero: e.target.checked }) )}
70       className="mr-2"
71     />
```

Les valeurs sont répercutées immédiatement sur l'affichage du prix total :

```
</div>
<div className="text-xl font-bold mt-4">Total: {total} €</div>
</div>
```

Cette logique supprime toute latence entre l'action et le retour visuel, améliorant fortement la perception de réactivité.

Cette approche connecte directement la logique front à Laravel via Inertia, garantissant la persistance des données et la cohérence du panier tout en maintenant un comportement de SPA fluide.

5) Sécurité front-end

L'UI (interface utilisateur) est pensée pour **empêcher l'injection** côté navigateur et **garantir** que toutes les règles de sécurité sont ré-appliquées côté serveur. Les points clés :

a) Anti-XSS par design (React)

Le rendu des champs saisis par l'utilisateur (**nom**, **numéro**, etc.) est **échappé par défaut** par React.

Aucun `dangerouslySetInnerHTML` n'est utilisé.

Exemple : fiche produit- seules des valeurs texte sont insérées et renvoyées au serveur via Inertia (pas d'HTML injecté).

Extrait : envoi des valeurs « brutes »

```
21  ✓  function handleAddToCart() {
22  ✓      router.post('/cart/add', {
23          maillot_id: maillot.id,
24          size: taille,
25          quantity: qte,
26          numero: personnalisation.numero ? numero : null,
27          nom: personnalisation.nom ? nom : null,
28  ✓      }, {
29          onSuccess: () => alert("Maillot ajouté au panier !"),
30      });
31  }
```

b) CSRF / intégration Inertia

Les **formulaires** envoyés par Inertia (router.post) bénéficient de la **protection CSRF de Laravel** (token géré par le middleware et le layout).

Aucun jeton n'est manipulé côté front, cela induit une **réduction du risque d'erreur**.

Fichier côté front : resources/js/Pages/LoginRegister.jsx

```
12  const { data, setData, post, processing, errors } = useForm({
13      login: "",
14      username: "",
15      password: "",
16      password_confirmation: "",
17  });
18
19
20  const handleChange = (e) => {
21      setData(e.target.name, e.target.value)
22  }
23
24  const handleSubmit = (e) => {
25      e.preventDefault()
26      post(isLogin ? "/login" : "/register")
27  }
28
```

c) Validation serveur systématique

Chaque entrée utilisateur est validée côté serveur (Laravel) : taille, quantité, formats, longueurs et ensembles de valeurs autorisées sont contrôlés avant tout traitement.

d) Autorisations / cloisonnement des données (403 si hors périmètre)

Avant modification d'un item de panier, on vérifie la propriété de l'élément. Si l'objet n'est pas reconnu, on obtient un code **403** immédiat. C'est une barrière simple et efficace pour empêcher l'accès horizontal (IDOR).

e) Réduction de surface d'exposition (données envoyées côté UI)

Dans la vue panier, on façonne les données renvoyées à React (mapping) pour exposer uniquement le nécessaire et recalculer le total côté serveur (source de vérité).

f) Authentification & hachage des mots de passe

À l'inscription, les mots de passe sont **hachés** (Hash::make) et les e-mails **validés** / **uniques**.

Bilan

L'interface React limite l'injection par échappement automatique, et n'insère pas d'HTML arbitraire. Les formulaires passent par Inertia (CSRF natif Laravel). Toutes les règles (tailles, quantités, formats, longueurs) sont revalidées côté serveur avec contrôle d'accès (403) et hachage des mots de passe. Le panier n'expose au client que les champs nécessaires, et les totaux sont recalculés côté back pour conserver la source de vérité.

3.5. Accessibilité et éco-conception

L'accessibilité a été prise en compte dès la conception de l'interface afin de garantir une expérience utilisateur inclusive, conformément aux recommandations du **RGAA (Référentiel Général d'Amélioration de l'Accessibilité)**.

1) Principes appliqués

Les principaux critères respectés sont :

- Navigation clavier :

Toutes les pages et formulaires sont accessibles à la navigation au clavier grâce aux attributs HTML natifs et à la gestion du focus dans les composants React (tabIndex, focus:ring de TailwindCSS). :

- **Contrastes** : Les couleurs ont été choisies pour offrir un contraste suffisant entre le texte et l'arrière-plan (minimum 4.5:1 pour le texte normal). J'ai effectué les vérifications avec l'outil intégré Lighthouse et le testeur RGAA.

- Structures sémantiques :

Les balises <header>, <main>, <section>, <footer> et <nav> ont été utilisées pour structurer les pages, permettant aux technologies d'assistance (lecteurs d'écran) de mieux comprendre la hiérarchie des contenus.

resources/js/Layouts/MainLayout.jsx utilisation de <main> (avec header/footer)

```
8      <Header />
9      <main className="flex-1">
10     {children}
11   </main>
12   <Footer />
```

resources/js/Pages/LoginRegister.jsx <main> + attributs ARIA pour une zone d'onglets

```
46   <main className="min-h-screen flex items-center justify-center bg-gradient-to-r from-purple-200 to-blue-200 py-12 px-4 sm:px-6 lg:px-8">
47     <div className="max-w-md w-full space-y-8">
48       { /* Tabs */ }
49       <div className="flex justify-center mb-6" role="tablist" aria-label="Navigation formulaire">
50         <button
51           onClick={() => setIsLogin(true)}
52           className={`px-4 py-2 font-medium text-lg ${isLogin ? "text-blue-600 border-b-2 border-blue-600" : "text-gray-500"}`}
53           role="tab"
54           aria-selected={isLogin}
55         >
56           Connexion
57         </button>
58         <button
59           onClick={() => setIsLogin(false)}
60           className={`px-4 py-2 font-medium text-lg ${!isLogin ? "text-blue-600 border-b-2 border-blue-600" : "text-gray-500"}`}
61           role="tab"
62           aria-selected={!isLogin}
63         >
64           Inscription
65         </button>
```

- Alternatives textuelles :

Les images des maillots disposent toutes d'un attribut alt descriptif.

Carte d'équipe (Home) – TeamCard dans resources/js/Pages/Home.jsx

```
252 <div className="relative rounded-lg overflow-hidden hover:transform hover:scale-105">
253   <img src={image} alt={team} className="w-full h-64 object-cover" />
254   <div className="absolute inset-0 bg-black/40 flex items-center justify-center">
```

Pour les cartes “nouveaux maillots” :

```
158   <img
159     src={maillot.image}
160     alt={`Maillot ${maillot.team}`}
161     className="w-full h-64 object-contain rounded-t-lg"
162   />
```

- Labels et formulaires :

Chaque champ de formulaire est associé à un `<label>` ou un attribut `aria-labelledby`. Les messages d'erreurs sont visibles et compréhensibles, tant visuellement qu'à la lecture vocale.

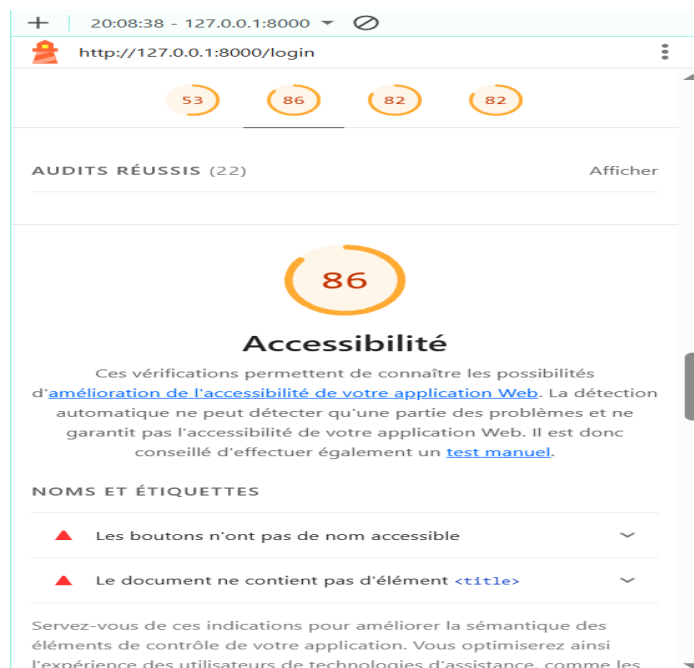
resources/js/Pages/LoginRegister.jsx -label lié par `htmlFor` + messages d'erreur

```
85   <label htmlFor="email" className="block text-sm font-medium text-gray-700">Email</label>
86   <input
87     id="email"
88     name="email"
89     type="email"
90     value={data.email}
91     onChange={handleChange}
92     required
93     className="w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm mt-1"
94   />
95   {errors.email && <p className="text-sm text-red-600">{errors.email}</p>}
96 </div>
97 </>
```

Navigation par onglets accessible (même fichier)

```
49 <div className="flex justify-center mb-6" role="tablist" aria-label="Navigation formulaire">
50   <button
51     onClick={() => setIsLogin(true)}
52     className={`px-4 py-2 font-medium text-lg ${isLogin ? "text-blue-600 border-b-2 border-blue-600"
53     role="tab"
54     aria-selected={isLogin}
55   >
56     Connexion
57   </button>
58   <button
59     onClick={() => setIsLogin(false)}
60     className={`px-4 py-2 font-medium text-lg ${!isLogin ? "text-blue-600 border-b-2 border-blue-600"
61     role="tab"
62     aria-selected={!isLogin}
63   >
64     Inscription
65   </button>
66 </div>
```

2) Lightscore page login (connexion/inscription)



Bilan

Le score **Lighthouse Accessibilité : 86/100** traduit une bonne conformité générale aux critères RGAA. Quelques points d'amélioration subsistent, notamment homogénéiser les contrastes dans certaines zones secondaires et renforcer les indications de focus sur les liens de navigation. Ces ajustements pourront être intégrés lors de la phase d'optimisation visuelle.

3) Compétences mobilisées :

J'ai appris à réaliser une interface utilisateur web statique et adaptable (prise en compte des normes d'accessibilité, design responsive). J'ai également pu développer une interface utilisateur web dynamique (intégration d'attributs ARIA, gestion de l'interactivité accessible).

4) Éco-conception

Optimisation des médias : conversion des images en formats légers (WebP) et compression systématique pour limiter le poids des pages.

Code CSS optimisé : grâce à **TailwindCSS**, les classes inutilisées sont supprimées lors de la phase de build (purgeCSS), ce qui réduit la taille du code distribué.

Chargement progressif des composants : avec Inertia.js, seuls les composants nécessaires sont chargés (lazy loading), ce qui limite la consommation de bande passante et améliore les performances côté client. Mon site de e-commerce profite du lazy chargement pour les images et pour les relations entre entités, ce qui améliore la rapidité et la réactivité.

5) Jeu d'essai fonctionnel

Fonctionnalité testée : Ajout au panier avec personnalisation (MaillotDetail.jsx)

N°	Cas de test	Données en entrée	Données attendues	Données obtenues	Résultat
1	Validation nom personnalisation	Nom="lolo2!" (chiffre + car. spécial)	Saisie bloquée, caractères invalides refusés	Input bloqué côté React (LOLO accepté)	✓
2	Personnalisation	Taille=L, Qté=1, Nom="Lolo", N°=10	Total recalculé avec suppléments	Total mis à jour instantanément	✓
3	Quantité dépasse le stock	Stock=25, saisie de 26	Quantité bloquée à 25, ajout impossible	Input limité à max={stockDispo}, recalé à 25	✓
4	N° personnalisation invalide	$1 \leq N^{\circ} \leq 99$	Saisie bloquée, valeur refusée	Input bloqué côté React ($1 \leq N^{\circ} \leq 99$)	✓
5	Non connecté — confirmation commande	Clic "Commander" depuis le panier	Redirection login/inscription	Redirection LoginRegister.jsx	✓

6) Veille sécurité front-end

Durant le projet, j'ai effectué une veille régulière sur les vulnérabilités front-end via **OWASP Top 10** et **CERT-FR**. Les points surveillés et appliqués :

Vulnérabilité	Source	Action appliquée
XSS (injection script)	OWASP A03	Échappement automatique React, pas de dangerouslySetInnerHTML
CSRF	OWASP A01	Token CSRF natif Laravel/Inertia
Broken Access Control	OWASP A01	Middleware auth, contrôle 403 côté serveur
Données sensibles exposées	OWASP A02	Mapping panier, exposition minimale des données au front

3.6. Bilan du front-end

La phase front-end du projet a permis de mettre en œuvre l'ensemble des compétences professionnelles CP1 à CP4 : installation et configuration de l'environnement de travail, maquettage des interfaces, réalisation statique puis dynamisation des pages utilisateur et administrateur.

Ce travail s'est traduit par :

- la création d'une interface moderne et responsive, respectant les bonnes pratiques de **conception web** et d'**ergonomie** ;
- l'intégration de fonctionnalités dynamiques (navigation fluide via Inertia.js, gestion des formulaires, mise à jour du panier en temps réel).
- l'application des recommandations en matière d'**accessibilité (RGAA :Référentiel Général d'Amélioration de l'Accessibilité)** et d'**éco-conception** (optimisation des images, réduction du code superflu, chargement progressif des composants).
- la mise en place de mécanismes garantissant la **sécurité** côté client (validation des entrées, protection CSRF via Laravel).

En résumé, mes pages front-end sont désormais en capacité d'interagir efficacement avec la partie back-end. Le front-end doit offrir aux utilisateurs finaux une expérience fluide, intuitive et sécurisée, conforme aux standards attendus dans une boutique en ligne moderne.

ANNEXES

2.2. Architecture de l'application

b) Shéma de fonctionnement

1. Requête utilisateur

Lorsqu'un utilisateur accède à une page (ex. /maillots), la route Laravel correspondante est appelée via le fichier routes/web.php.

Exemple :

```
// Routes pour les maillots
Route::get('/maillots/{id}', [MaillotController::class, 'show']->name('maillots.show'));
```

2. Traitement côté serveur

Le contrôleur exécute la logique métier (récupération des maillots, pagination, filtrage par club, etc.), via les modèles Eloquent :

MaillotController.php

```
14      $maillot = Maillot::with('club')->findOrFail($id);
15
16      // ajouter les tailles, quantités
17      $tailles = ['S', 'M', 'L', 'XL'];
18      $quantite = 1200; // ou récupère depuis la BDD
19
20      return Inertia::render('MaillotDetail', [
21          'maillot' => $maillot,
22          'tailles' => $tailles,
23          'quantite' => $quantite,
24          'nom' => $maillot->nom,
25          'numero' => $maillot->numero,
26          'prix' => 20,
27          'prix_nom' => 3,
28          'prix_numero' => 2,
29      ]);
```

3. Transmission des données via Inertia.js

Inertia agit comme une “passerelle” entre Laravel et React.

Les données envoyées par le contrôleur sont automatiquement transformées en props React et injectées dans le composant MaillotsList.jsx.

4. Rendu et interactions côté client (React)

Le composant React affiche les données et gère l’interactivité (filtrage, clics, ajouts au panier...).

Toute action utilisateur (soumission de formulaire, suppression d’article, etc.) est renvoyée vers les routes Laravel via Inertia, sans rechargement complet de page :

```
21  function handleAddToCart() {
22      router.post('/cart/add', {
23          maillot_id: maillot.id,
24          size: taille,
25          quantity: qte,
26          numero: personnalisation.numero ? numero : null,
27          nom: personnalisation.nom ? nom : null,
28      }, {
29          onSuccess: () => alert("Maillot ajouté au panier !"),
30      });
31  }
```

5. Mise à jour dynamique

Inertia ne recharge que le composant concerné, simulant un comportement SPA (Single Page Application) tout en restant dans un monolithe Laravel unique.

Cela offre la possibilité d’obtenir une navigation fluide, la cohérence des sessions Laravel (authentification, middleware) et une sécurité unifiée.

6. Les avantages de cette approche

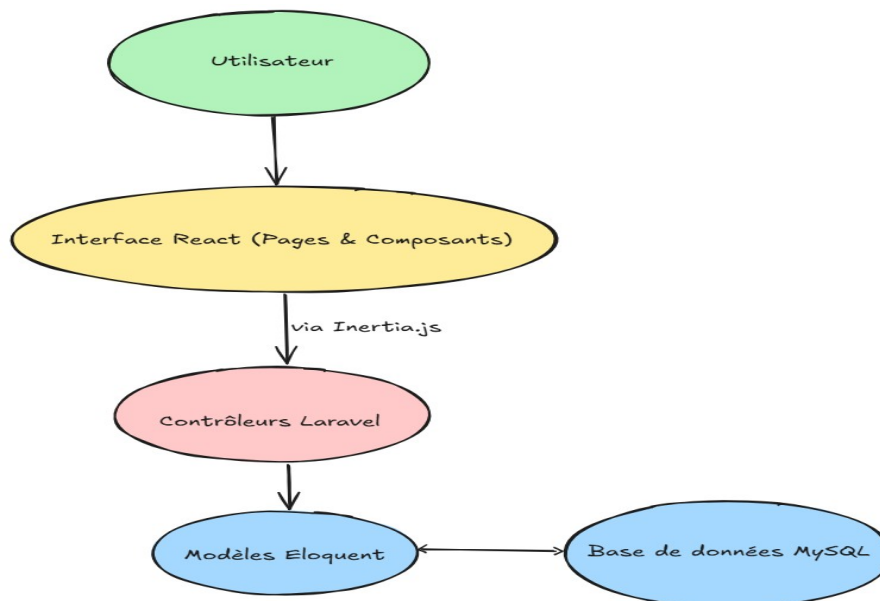
Simplicité de déploiement (une seule application à héberger), Sécurité renforcée, Performances (Vite et React assurent un rendu rapide et interactif), Cohérence des données (Laravel gère la logique métier et la validation, React se concentre sur l’affichage).

Bilan

Ce choix architectural permet de bénéficier de la souplesse du front React tout en conservant la stabilité et la sécurité d'un framework back-end complet.

Laravel centralise les données, la validation et les règles métier, tandis qu'Inertia.js fait office de passerelle vers une interface fluide et moderne.

c) Schéma global d'architecture



2.3 Qualité et sécurité

- **Authentification** : le middleware auth protège l'accès aux zones privées (compte, adresses, panier). Seuls les utilisateurs connectés peuvent accéder à ces fonctionnalités.

```
app > Http > Middleware > AuthSessionMiddleware.php > ...
6 use Illuminate\Http\Request;
7 use Symfony\Component\HttpFoundation\Response;
8
9 class AuthSessionMiddleware
10 {
11     /**
12      * Handle an incoming request.
13      *
14      * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
15      */
16     public function handle(Request $request, Closure $next): Response
17     {
18         $sessionId = $request->cookie('session-id');
19
20         if (!$sessionId) {
21             return redirect()->route('Login');
22         }
23
24         $session = \App\Models\UserSession::with('user')
25             ->where('id', $sessionId)
26             ->where('expires_at', '>', now())
27             ->first();
28
29         if (!$session) {
30             return redirect()->route('Login')->withoutCookie('session-id');
31         }
32     }
33 }
```

- **Sessions utilisateurs** : un suivi des connexions est assuré grâce à la table dédiée `user_sessions`, permettant de tracer les activités et de renforcer le contrôle d'accès.

```
app > Models > UserSession.php > UserSession
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6  use Illuminate\Database\Eloquent\Relations\BelongsTo;
7
8  class UserSession extends Model
9  {
10     protected $fillable = [
11         'id',
12         'user_id',
13         'expires_at',
14         'ip_address',
15         'user_agent',
16         'last_activity',
17     ];
18
19     protected $casts = [
20         'expires_at' => 'datetime',
21         'last_activity' => 'datetime',
22         'created_at' => 'datetime',
23     ];
24
25     public $incrementing = false;
26     protected $keyType = 'string';
27 }
```

- **Séparation des responsabilités** : l'architecture du projet suit le principe *Separation of Concerns*. Les contrôleurs gèrent la logique applicative, les modèles manipulent les données, et les composants React/Tailwind assurent la présentation. Cette structuration rend le projet plus facile à maintenir et à tester. En combinant mécanismes de sécurité natifs et bonnes pratiques de développement, l'application bénéficie d'un socle fiable, sécurisé et durable, garantissant à la fois la protection des données et la facilité d'évolution du projet.

Chapitre 3 -Développement de la partie front-end sécurisée

3.1. CP1 -Installer et configurer l'environnement de travail

Etapes d'installations :

1- Clonage du dépôt git

git clone https://github.com/utilisateur/Lam_Maillot_de_Foot.git

cd Lam_Maillot_de_Foot

2- Configuration des variables d'environnement

```
cp .env.example .env
```

Personnalisation des accès base de données

3- Installer des dépendances Back-end :

```
composer install
```

4- Installation des dépendances front-end :

```
npm install
```

5- Génération de la clé d'application Laraval :

```
php artisan key : generate
```

6- Migration et alimentation initiale de la base :

```
php artisan migrate --seed
```

7- Lancement des serveurs de développement :

```
php artisan serve (serveur)
```

```
npm run dev : build et hot reload côté front
```

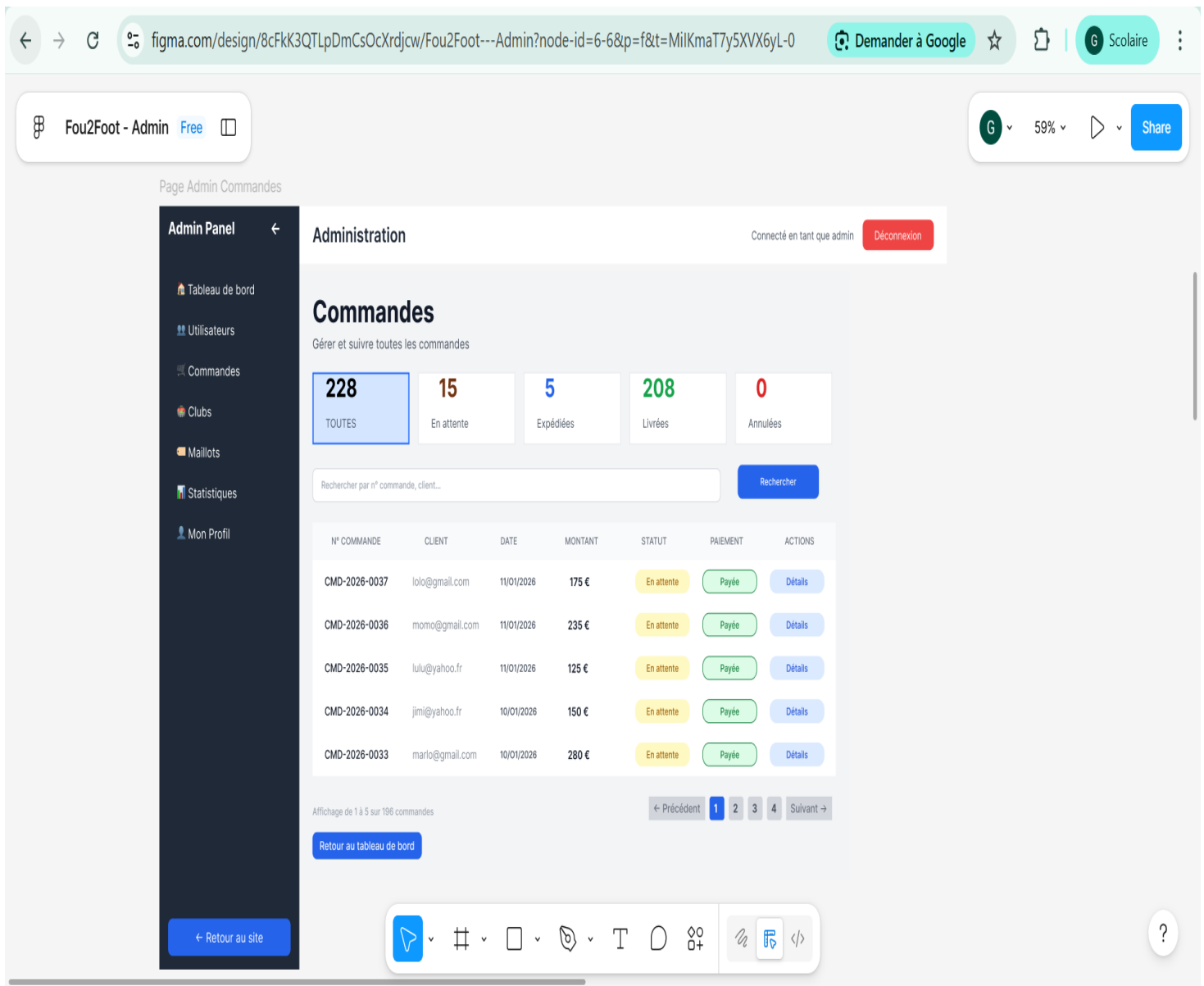
En production, la commande suivante est utilisée : `npm run build`

Les scripts npm permettent de piloter la compilation du front-end :

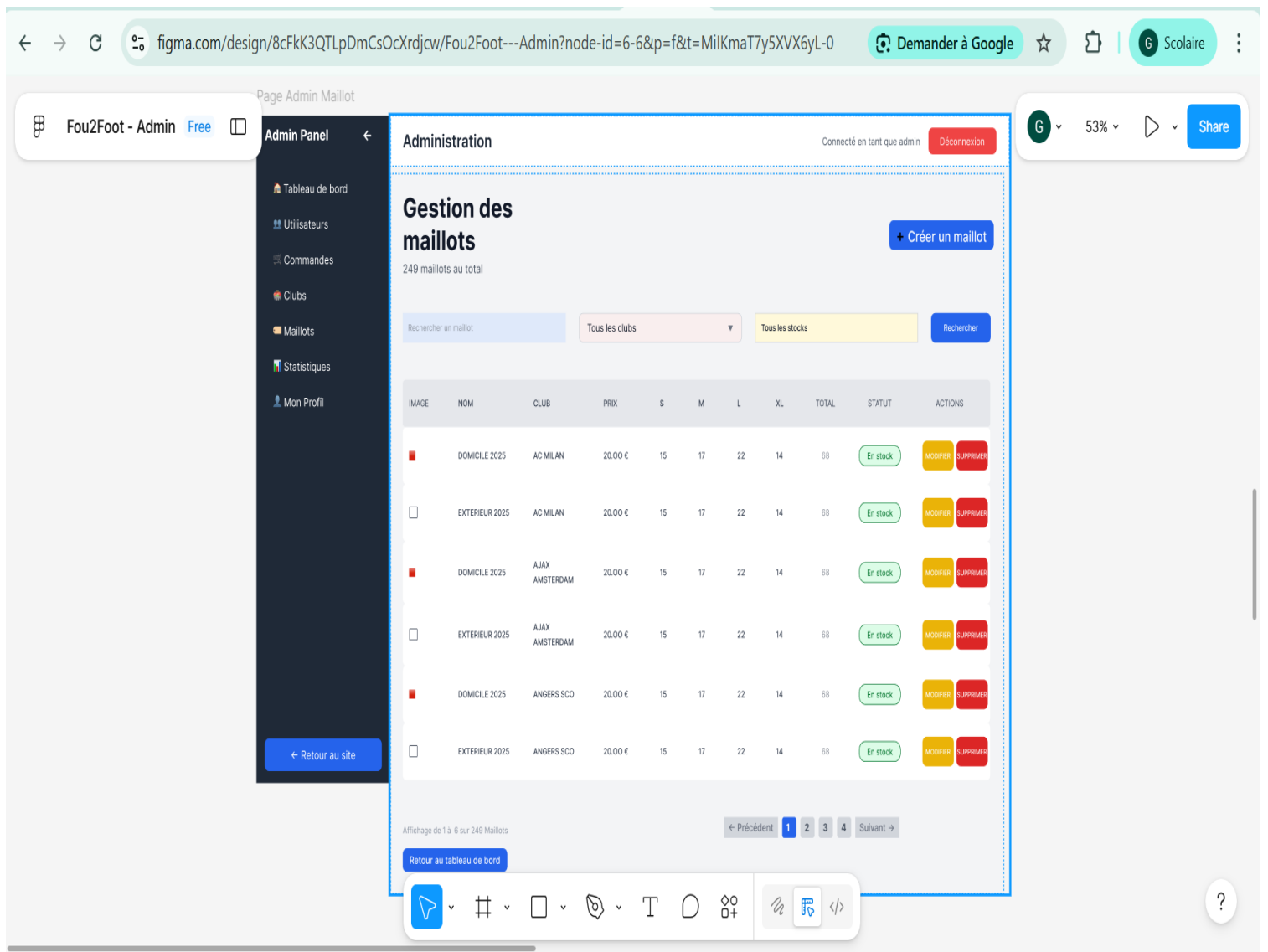
- `npm run dev` lance le serveur de développement avec **HMR** (Hot Module Replacement), offrant un rechargement instantané et fluide.
- `npm run build` génère les fichiers optimisés pour la production, garantissant des performances maximales grâce à la minification et à la suppression des classes inutilisées.

3.2. CP2 -Maquetter une application

Gestion des commandes : tableau avec statuts colorés (en attente, expédiée, livrée, annulée), filtres par date et recherche.



Gestion des maillots : catalogue admin avec prix, stock, actions de modification.



Modale créer un maillot

Créer un maillot

Club

Sélectionner un club

Nom du maillot

Ex: Domicile 2024

Prix (€)

20.00

Stocks par taille

S M L XL

10 10 10 10

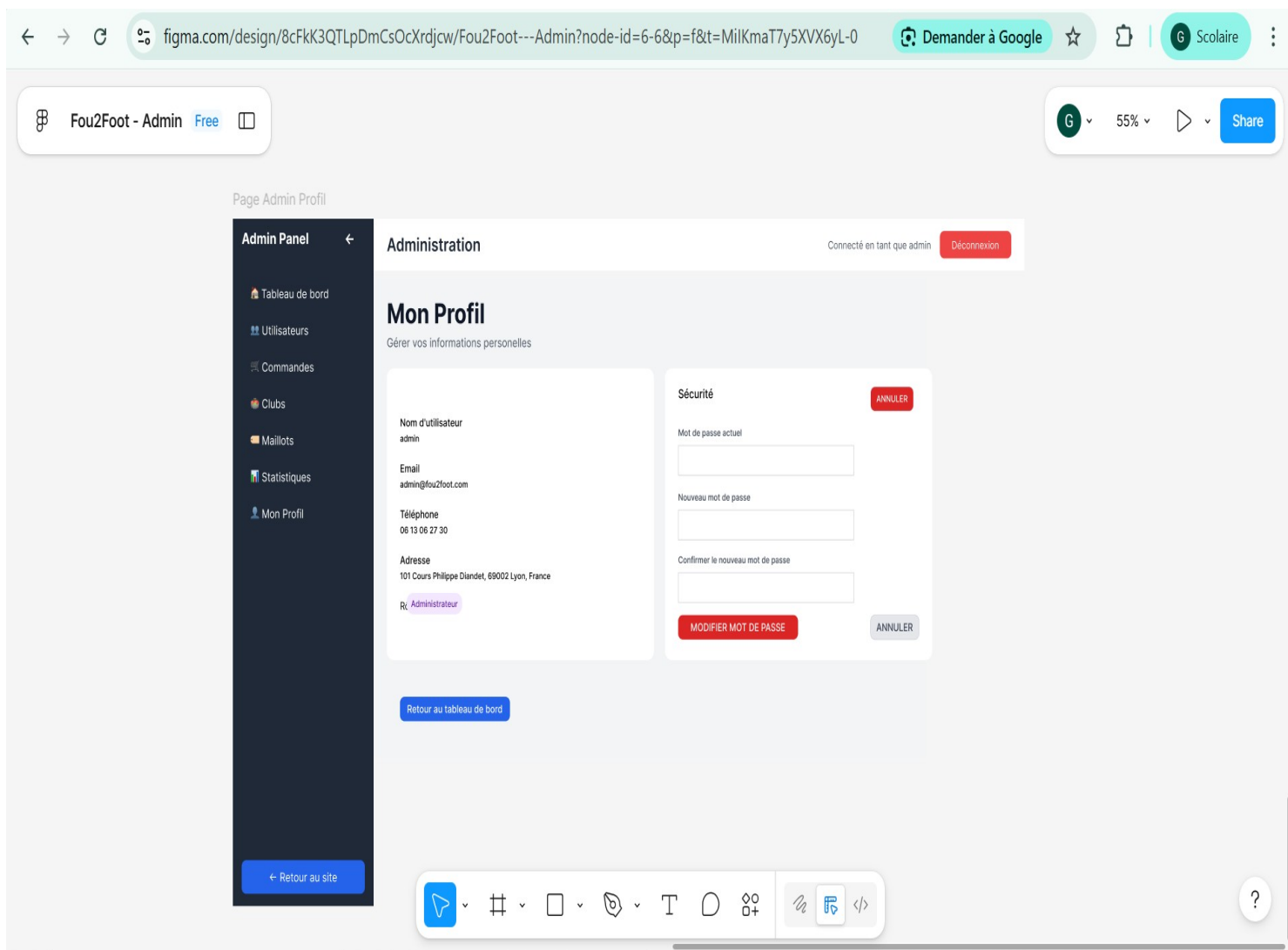
Image

Choisir un fichier

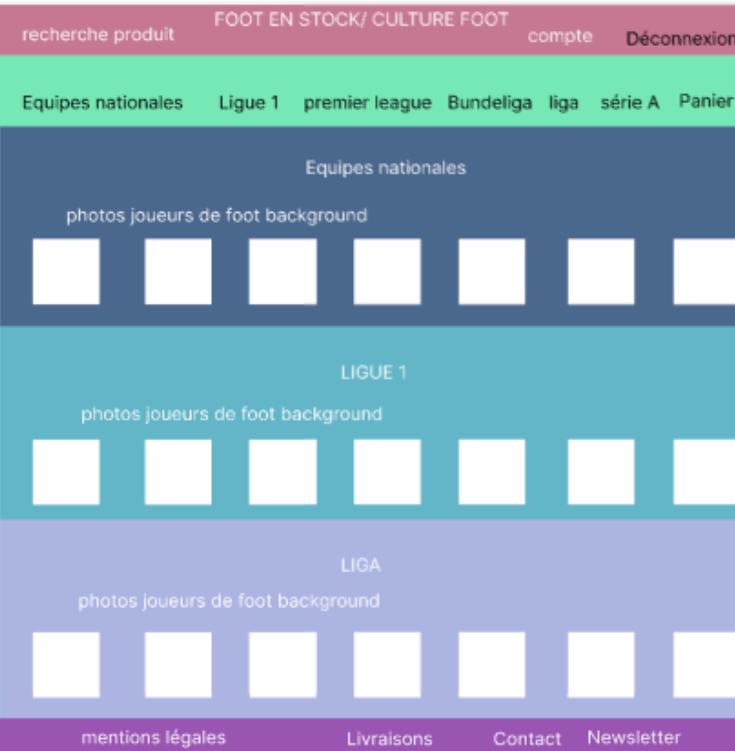
Annuler Créer

448 x 572 Hug

Profil administrateur : informations personnelles et modification du mot de passe.



WIREFRAME PAGE ACCUEIL



WIREFRAME Connexion/ Inscription

Frame 27

recherche produit FOOT EN STOCK/ CULTURE FOOT compte Déconnexion

Equipes nationales Ligue 1 premier league Bundeliga liga série A Panier

S'enregistrer

email

identifiant

Mdp

se souvenir de moi ☐

Valider

Connexion

identifiant ou email

Mdp

Valider

Frame 34

mentions légales Livraisons Contact Newsletter

WIREFRAME Tableau de bord

Frame 30

recherche produit FOOT EN STOCK/ CULTURE FOOT compte Déconnexion

Equipes nationales Ligue 1 premier league Bundeliga liga série A Panier

NAVIGATION

Ma Commande

Adresses

Détails du compte

Ma wishlist

Déconnexion

TABLEAU DE BORD

Ma Commande

Adresses

Détails du compte

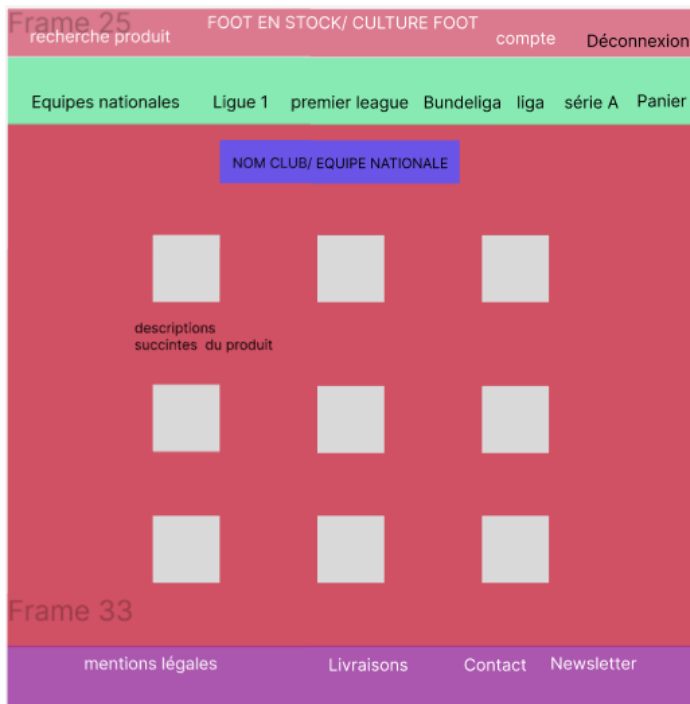
Ma wishlist

Activités récentes

Frame 31

mentions légales Livraisons Contact Newsletter

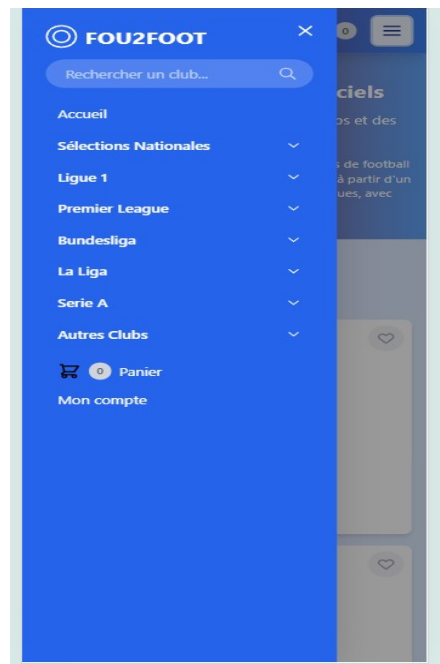
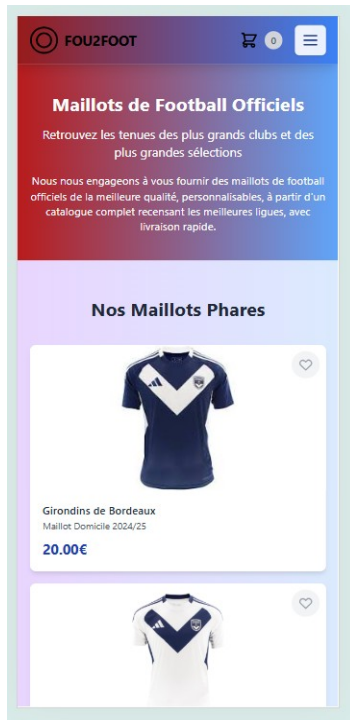
WIREFRAME Liste Maillots



WIREFRAME Détail d'un maillot

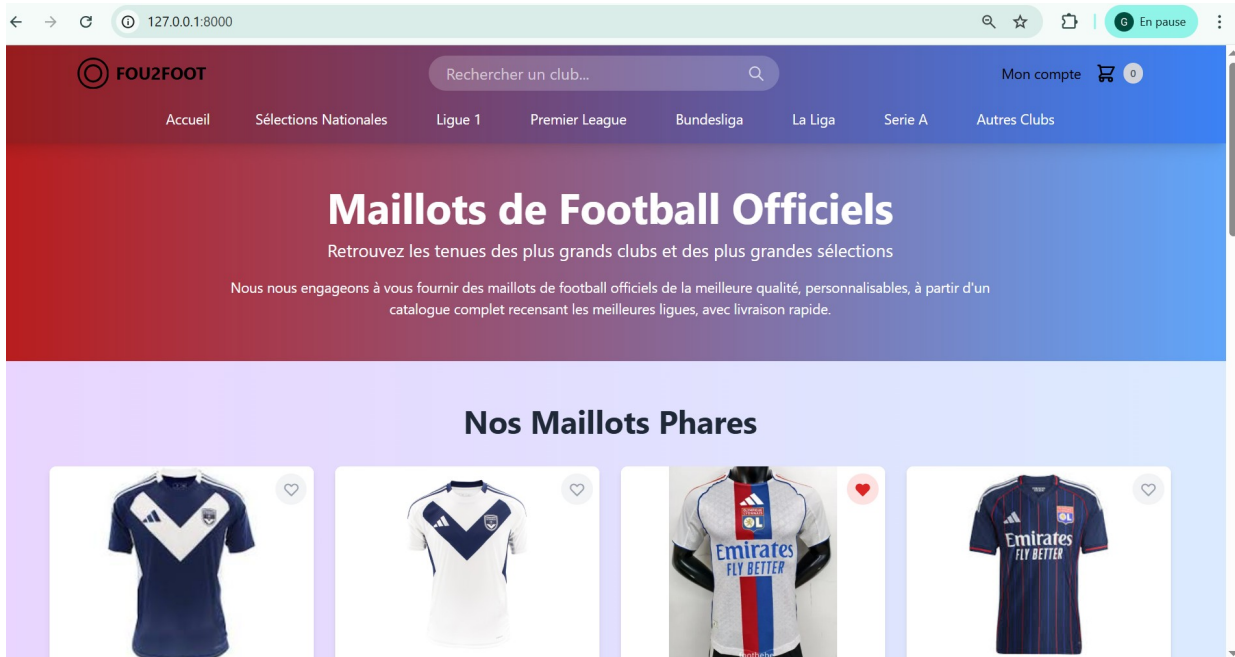


Design responsive avec menu burger (Pixel7)



On peut simuler ici la navigation partielle d'un utilisateur à partir des pages front-end.

Page Accueil (ne nécessite pas de connexion car c'est la vitrine du site)



Page inscription

Page connexion

Connexion

Inscription

Nom d'utilisateur ou Email

Mot de passe

Se connecter

 Fou2Foot

Informations

Service client

Nous suivre

Tableau de bord

← → ↺ 127.0.0.1:8000/dashboard

🔍 Scolaire

 FOU2FOOT

Rechercher un club...

Mon compte Déconnexion  7

Accueil

Sélections Nationales

Ligue 1

Premier League

Bundesliga

Liga

Série A

Autres

Navigation

 Tableau de bord

 Commandes

 Adresse

 Détails du compte

 Ma wishlist

 Se déconnecter

Bienvenue, mina !

Votre Email : mina@carp.com

Mon Tableau de Bord

 **Mes Commandes**

Suivez vos achats passés et en cours.

 **Adresse**

Gérez votre adresse de livraison.

 **Détails du compte**

Modifiez vos informations personnelles.

 **Ma wishlist**

Retrouvez vos articles favoris.

Activité récente

Détails du compte

127.0.0.1:8000/accountdetails

Bienvenue, mina !
Votre Email : mina@carp.com

Mon compte

Gérez vos informations personnelles et vos préférences

Informations personnelles

[Modifier](#)

Identifiant
mina

Nom complet
MINA BELLA

Email
mina@carp.com

Téléphone
77 33 11 88 66

Sécurité

[Changer le mot de passe](#)

Adresses

127.0.0.1:8000/addresses

FOU2FOOT

Rechercher un club...

Mon compte Déconnexion 7

Accueil Sélections Nationales Ligue 1 Premier League Bundesliga Liga Série A Autres

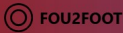
Mes adresses

Adresse de facturation

Facturation Par défaut

MINA BELLA
rue gandolphe
06210 Mandelieu
FR
77 33 11 88 66
[Modifier](#) [Supprimer](#)

Liste Maillots d'un club



Rechercher un club...

Mon compte  0

AccueilSélections NationalesLigue 1Premier LeagueBundesligaLa LigaSerie AAutres Clubs



Maillots de Olympique Lyonnais

Découvrez notre collection de maillots Olympique Lyonnais - 10 maillots disponibles



OL Domicile 2024

[Voir les détails >](#)



OL Extérieur 2024

[Voir les détails >](#)



maillot third 2024

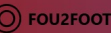
[Voir les détails >](#)



OL 75 ans

[Voir les détails >](#)

Présentation Maillot



Rechercher un club...

Mon compte  0

AccueilSélections NationalesLigue 1Premier LeagueBundesligaLa LigaSerie AAutres Clubs





Olympique Lyonnais

OL Domicile 2024

Prix : 20.00 €

Type : Maillot

Stock total : 93 (Taille S : 25 disponibles)

Taille : S - 25 dispo

Quantité :

☐ Ajouter un numéro (+2 €)

☐ Ajouter un nom (+3 €)

Total : 20 €

AJOUTER AU PANIER

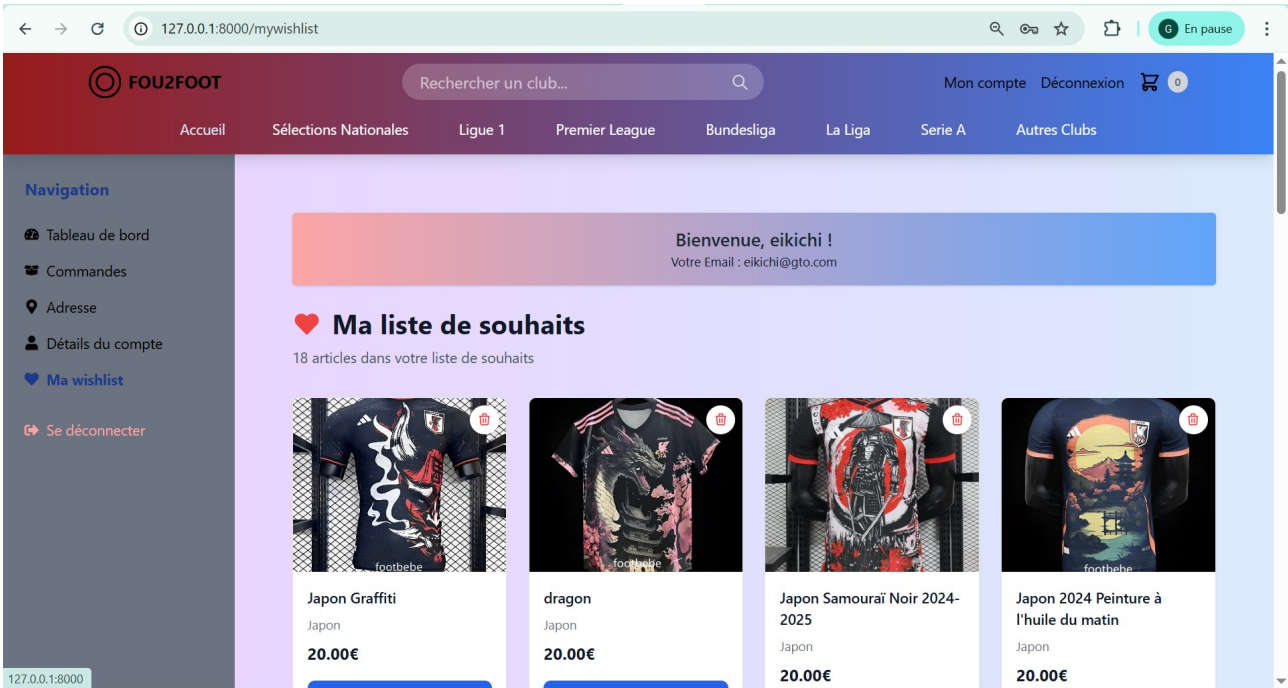


Informations

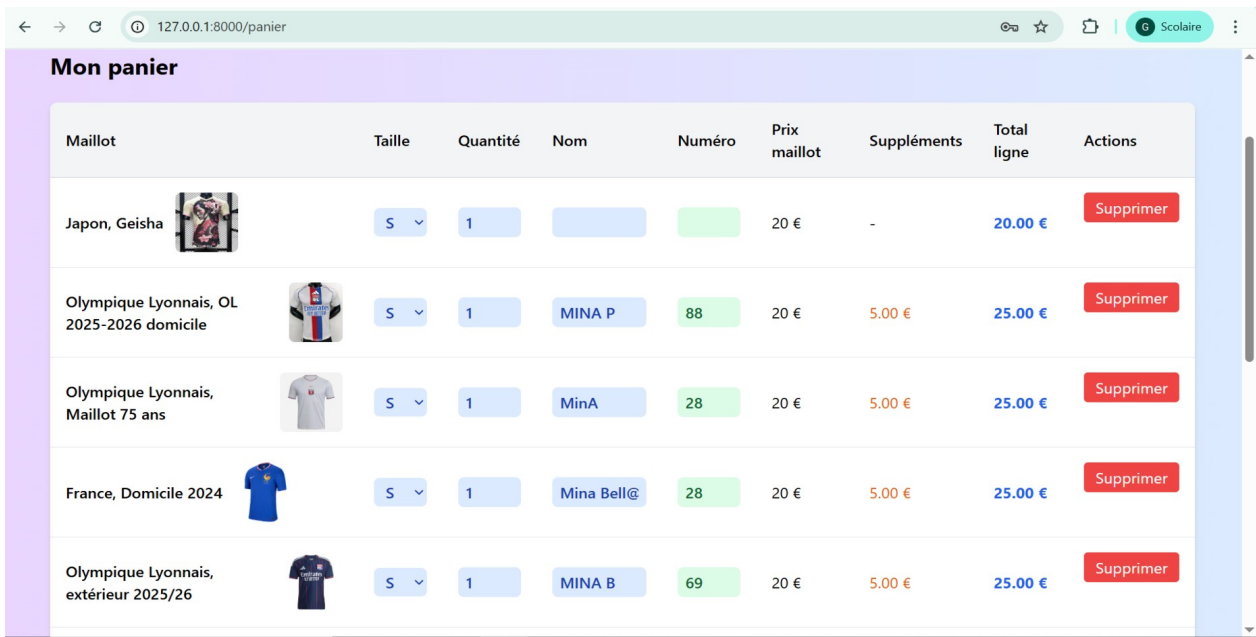
Service client

Nous suivre

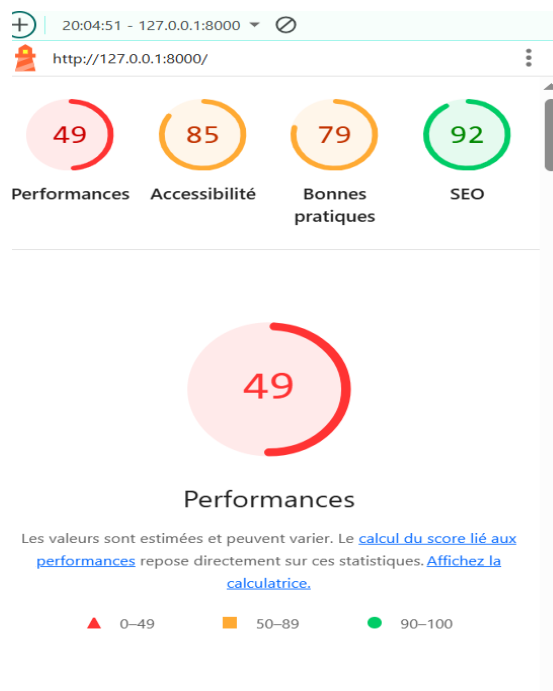
Page WishList



Panier de l'utilisateur



Score lighthouse accueil



Interprétation et axes d'amélioration page accueil

Performances (49)

Le score moyen est lié à l'exécution en environnement de développement. En **mode production** (npm run build), les fichiers sont minifiés et purgés par **Vite** et **TailwindCSS**, ce qui réduit considérablement le poids des ressources. Le déploiement sur un serveur **Nginx** avec **compression gzip** et **mise en cache** devrait améliorer le score au-delà de 85.

Accessibilité (85)

Le site répond déjà à plusieurs critères du **RGAA** : navigation clavier fluide, contraste suffisant, structure sémantique cohérente (header, main, footer), alt descriptifs sur les images, **aria-label** sur les icônes.

Les axes d'amélioration concernent le renforcement du focus sur certains éléments interactifs et la hiérarchisation des titres (h1 à transformer en h3).

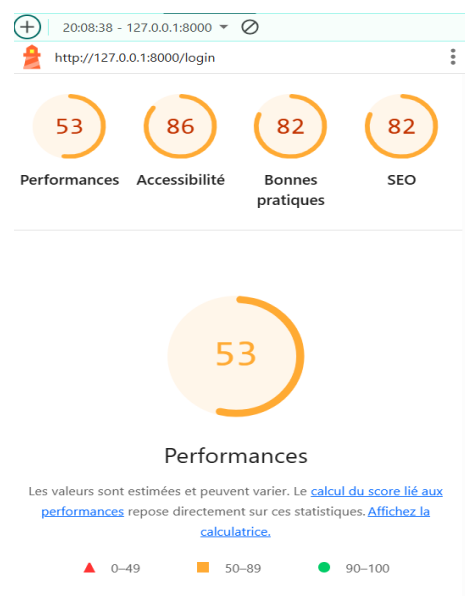
Bonnes pratiques (79)

Quelques recommandations de **Lighthouse** concernent la mise à jour de dépendances et la sécurisation de liens externes. Ces points seraient corrigés lors du passage en production.

SEO (92)

Les balises structurantes (title, description, lang) et la hiérarchie du contenu permettent une très bonne indexabilité du site. L'ajout d'un sitemap XML et de balises OpenGraph pourrait encore l'améliorer.

Score lighthouse connexion/ inscription



Interprétation et axes d'amélioration page connexion /inscription

Performances (53)

Ce score reste satisfaisant en phase de développement. Les principales améliorations à prévoir :

- exécuter un **build de production** (`npm run build`) pour minifier les fichiers JS et CSS,
- activer la **mise en cache du navigateur**,
- compresser les ressources statiques via **gzip** sur le serveur (Nginx ou Apache).

Ces optimisations devraient porter le score de performance au-delà de 85 en production.

Accessibilité (86)

Les formulaires de connexion et d'inscription respectent les critères RGAA :

- tous les champs sont associés à un label,
- le focus clavier est visible,

- Les erreurs de validation sont signalées en texte clair.

Une légère amélioration du contraste sur certains boutons renforcerait la lisibilité.

Bonnes pratiques (82)

L'audit recommande :

- de mettre à jour certaines dépendances NPM.
- d'ajouter `rel="noopener noreferrer"` aux liens externes,
- d'éviter les images trop volumineuses.

SEO (82)

Le SEO est solide pour une page de type « authentification », même s'il est vrai que cette page n'est généralement pas indexée. L'ajout de métadonnées cohérentes et d'un `robots.txt` adapté renforcerait la cohérence SEO globale du site.

3.4. CP4 -Développer une interface utilisateur dynamique

1) Navigation dynamique

La route correspondante est définie dans `routes/web.php`

```
74 // Routes pour les maillots
75 Route::get('/maillots/{id}', [MaillotController::class, 'show'])->name('maillots.show');
76
77
```

Et la méthode `show()` du `MaillotController` prépare les données avant l'envoi à React :

```
10 class MaillotController extends Controller
11 {
12     public function show($id)
13     {
14         $maillot = Maillot::with('club')->findOrFail($id);
15
16         // ajouter les tailles, quantités
17         $tailles = ['S', 'M', 'L', 'XL'];
18         $quantite = 1200; // ou récupère depuis la BDD
19
20         return Inertia::render('MaillotDetail', [
21             'maillot' => $maillot,
22             'tailles' => $tailles,
23             'quantite' => $quantite,
24             'nom' => $maillot->nom,
25             'numero' => $maillot->numero,
26             'prix' => 20,
27             'prix_nom' => 3,
28             'prix_numero' => 2,
29         ]);
30     }
31 }
```

Ainsi, le passage entre la liste des produits et la fiche détail s'effectue instantanément, sans rupture de navigation.

2) Interaction avec le panier

Ajout au panier depuis la page MaillotDetail.jsx. Lorsque l'utilisateur choisit la taille et la quantité, le composant envoie une requête Inertia vers Laravel (voir extrait code dans chapitre correspondant):

Cette requête appelle la route définie dans routes/web.php

```
92 Route::get('/panier', [CartController::class, 'show'])->name('cart.show');
93 Route::post('/cart/add', [CartController::class, 'add'])->name('cart.add');
```

Et le contrôleur CartController.php crée ou met à jour l'article dans la base de données :

```
95 }
96 public function add(Request $request)
97 {
98     $cart = Cart::firstOrCreate(['user_id' => Auth::id()]);
99
100     $item = $cart->items()
101         ->where('maillot_id', $request->maillot_id)
102         ->where('size', $request->size)
103         ->where('numero', $request->numero ?? '')
104         ->where('nom', $request->nom ?? '')
105         ->first();
106
107     if ($item) {
108         $item->increment('quantity', $request->quantity);
109     } else {
110         $cart->items()->create([
111             'maillot_id' => $request->maillot_id,
112             'size' => $request->size,
113             'quantity' => $request->quantity,
114             'numero' => $request->numero,
115             'nom' => $request->nom,
116         ]);
117     }
118
119     return redirect()->route('cart.show')->with('success', 'Article ajouté au panier');
120 }
121 }
```

L'utilisateur voit immédiatement le panier mis à jour, sans rechargement complet, grâce à **Inertia.js**.

